

Modelling structured trial-by-trial variability in evidence accumulation

Steven Miletić, Niek Stevenson, Luke Strickland, Andrew Heathcote

21 May, 2026

Introduction

In this tutorial, we will demonstrate how to design dynamic evidence accumulation models and fit them to data using hierarchical Bayesian methods. We will utilize the *EMC2* package, assuming some prior knowledge of its functionalities (for reference, see the *EMC2* tutorial (Stevenson et al. 2024)). Specifically, we will fit models to dataset 1 from (Miletić et al. 2025), which corresponds to the 'choice, short [CS]' condition from (Wagenmakers, Farrell, and Ratcliff 2004). In this experiment, participants were presented with a single digit and required to determine whether the digit was even or odd.

First, let's load the required packages and data:

```
# rm(list=ls())
# TMP -- install the right branch
# remotes::install_github("ampl-psych/EMC2@dev", dependencies=TRUE, Ncpus=8,
# configure.args=c('--native')); rs.restartR()
# END TMP
library(EMC2)
set.seed(1) # for reproducibility

# This is some code that contains plotting functions used later on in this tutorial
code_url <- paste0(
  "https://raw.githubusercontent.com/StevenM1/",
  "dynamic_tutorial/refs/heads/main/plotting_utils.R")
source(code_url)

# Load in the data
data_url <- paste0(
  "https://github.com/StevenM1/dynamic_tutorial/",
  "raw/refs/heads/main/datasets/dataset_1.RData")
load(url(data_url))

# Inspect the data
print(head(dat))
#>   subjects      S      R      rt
#> 1         1 even even 0.542
#> 2         1 even even 0.426
#> 3         1 even even 0.501
#> 4         1 even even 0.403
#> 5         1 odd  odd 0.497
#> 6         1 even even 0.518
```

Here, **subjects** refers to subject number, **S** to the presented stimulus (even/odd), **R** the response (even/odd), and **rt** the response time in seconds.

Baseline models

We begin by setting up the 'static' baseline models without structured trial-by-trial variability. For a comprehensive tutorial on specifying designs in *EMC2*, please refer to the *EMC2* tutoial (Stevenson et al. 2024). Below, we provide a basic set-up of two baseline models: first using a racing diffusion model (RDM), and then using a Wiener diffusion model (WDM), which is the diffusion decision model (DDM) without between-trial variabilities (sometimes called the 'simple' DDM). With these two baseline models, we first demonstrate a full analysis pipeline, including model specification, prior specification, fitting, posterior predictive checks, and model comparisons, and a simpel recovery. Afterwards, we will demonstrate how model specifications can be altered to include time-varying parameters. The other functionality -- fitting routines, posterior predictives checks, model comparisons, parameter recoveries -- follow the same logic.

In *EMC2*, model specification is done via the `design()` function. It minimally requires the data (or, alternatively, manual specification of the factors of interest in data and their levels), the model (e.g., `RDM`, `DDM`), and a formula specifying which model parameters vary with which design factor using *R*'s linear modelling language. The present data only have one design factor `S` (for stimulus) with two levels: odd and even. *EMC2* always uses response coding of accumulators (in race models) and bounds (in the WDM/DDM). For this reason, DDM's the drift rate should vary with `S`, since the evidence for an 'odd' response is different in trials with an odd number versus an even number. We can enforce its value to be the same magnitude but opposite sign in odd versus even trials by specifying a contrast matrix. When a contrast matrix is specified, the column names represent *sampled* parameters, and the matrix values indicate how the sampled parameters are linearly combined to calculate the model parameters on each design cell (rows). As an example, we define `Smat` here with two rows (corresponding to the factor levels):

```

Smat <- matrix(c(-1,1), nrow=2, dimnames=list(NULL,"dif"))
design_WDM <- design(model=DDM,
                    data=dat,
                    contrasts=list(S=Smat),
# The documentation in ?DDM lists all the parameters and their estimation scales
                    formula=list(Z~1, v~S, a~1, t0~1))

#>
#> Sampled Parameters:
#> [1] "Z"      "v"      "v_Sdif" "a"      "t0"
#>
#> Design Matrices:
#> $Z
#> Z
#> 1
#>
#> $v
#> S v v_Sdif
#> even 1 -1
#> odd 1 1
#>
#> $a
#> a
#> 1
#>
#> $t0
#> t0
#> 1
#>
#> $s
#> s
#> 1
#>
#> $st0
#> st0
#> 1
#>
#> $sv
#> sv
#> 1
#>
#> $SZ
#> SZ
#> 1

```

This process of combining *sampled* parameters into *estimated* parameters is what we call *design mapping*, and it will play an important role later when specifying trends. The design matrices are by default printed by `design()`; but `mapped_pars()` gives some additional information:

```

mapped_pars(design_WDM)
#> $v
#> S
#> even : v - v_Sdif
#> odd  : v + v_Sdif

```

Note how, following R's linear modelling language, an intercept is automatically added for drift rate v . In this case, the *sampled* parameter v corresponds to an overall drift bias towards the upper bound. We can choose to set this as a constant to 0 by adding a `constants` argument to `design()`, as we will do in later

examples below.

For race models like the RDM, *EMC2* creates a data-augmented design matrix (**dadm**) under the hood - a data frame with one row *per accumulator* per trial per subject. **dadms** always have a factor column **lR** to indicate which *response* each accumulator codes for ('odd', 'even' in this case). Additionally, when provided a function that determines what response is correct (the **matchfun**), a second factor column **lM** will be added to indicate whether the accumulator matches the correct response. This is useful, because this will allow us to estimate the two accumulator drift rates with a mean-difference parameterisation: One parameter for the mean evidence, and one for the difference in evidence. We do this by setting up a contrast matrix **ADmat** with values -0.5, 0.5 corresponding to the mismatching and matching accumulator rows, respectively. Combined:

```
# set up contrast matrix for mean-difference parameterisation
ADmat <- matrix(c(-.5,.5), ncol=1, dimnames=list(NULL,'d'))

design_RDM <- design(model=RDM,
                    data=dat,
                    contrasts=list(lM=ADmat),
                    matchfun=function(d) d$S==d$lR,
                    formula=list(B ~ 1, v ~ lM, t0 ~ 1))

#>
#> Sampled Parameters:
#> [1] "B"      "v"      "v_lMd"  "t0"
#>
#> Design Matrices:
#> $B
#> B
#> 1
#>
#> $v
#>      lM v v_lMd
#> TRUE 1  0.5
#> FALSE 1 -0.5
#>
#> $t0
#> t0
#> 1
#>
#> $A
#> A
#> 1
#>
#> $s
#> s
#> 1
```

Note that the design matrix for the drift rates **v** looks similar as in the case of the DDM, but the rows now correspond to the matching and mismatching accumulator, instead of the two stimulus levels. We can again inspect the **mapped_pars()**:

```
mapped_pars(design_RDM)
#> $v
#>      lM
#> TRUE   : exp(v + 0.5 * v_lMd)
#> FALSE  : exp(v - 0.5 * v_lMd)
```

Which again looks similar, but also shows something perhaps unexpected: The linear combination of **v** and **v_lMd** is exponentiated. This is default behavior of *EMC2*: Parameters that are strictly positive (such as

RDM's drift rates, and thresholds and non-decision times in both the DDM and RDM) are estimated on the log-scale, since negative values have either no known analytical likelihood (RDM drift rate) or the likelihood is 0 (thresholds and non-decision times). Additionally, some parameters (such as the DDM's start point relative to its bound) is estimated on the probit scale to enforce that, after transformation, it is between 0 and 1. The default transformations can be found in the help functions of the models `?DDM` and `?RDM` (as well as `?LBA` and `?LNR`), and can be overridden by providing a custom transform to `design()` using the `transforms` argument. We will provide some demonstrations below, but for now, we wish to highlight that *sampled* parameters are first *mapped* to design cells and then *transformed* to arrive at the final design-cell-wise parameter value that is used to calculate a data point's likelihood. This is important because it has implications for specifying time-varying parameters.

EMC2 uses a particle-within-Gibbs sampler, which commits to a multivariate Gaussian group-level distribution to facilitate Gibbs sampling. This also restricts the options to set priors to a multivariate normal as well. One benefit of the transforms is that the implied prior domain always excludes impossible regions. The default priors for all parameters is $N(0,1)$ on the sampled scale. The `plot_prior()` function can be used to visualise the corresponding *implied* prior on the natural scale:

```
prior_RDM <- prior(design_RDM)
plot(prior_RDM, layout=c(1,4))
```

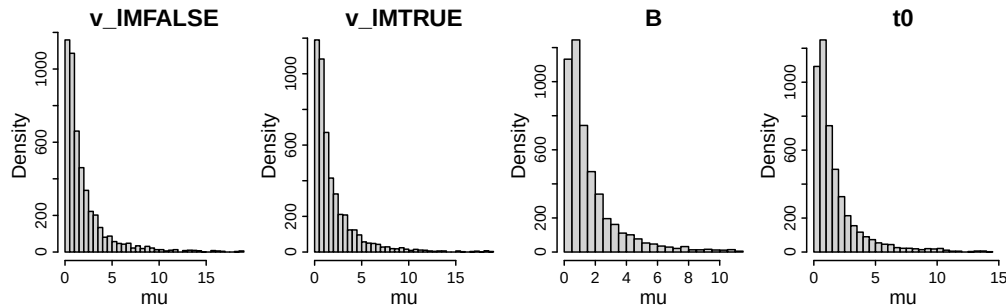


Figure 1: Implied default priors for the group-level mean parameters of the baseline RDM model. `v_1MTRUE` and `v_1MFALSE` refer to the drift rate of the accumulators matching ('correct') and mismatching ('error') the stimulus, respectively; `B` to the threshold, and `t0` to the non-decision time.

Priors can be overridden with the `prior()` function, as documented in `?prior` and demonstrated in examples below. Next, we can combine the data, design, and priors, and fit:

```
emc_rdm <- make_emc(dat, design_RDM, prior_list=prior_RDM)
emc_wdm <- make_emc(dat, design_WDM) # when not specified, the default priors are used

emc_rdm <- fit(emc_rdm)
emc_wdm <- fit(emc_wdm)
```

After fitting, it is important to check the quality of convergence. The `check()` function plots the group-level mean, group-level variance, and subject-level parameters with the highest R-hat (i.e., the worst cases), and it prints convergence statistics (Rhat, ESS) to the console:

```
check(emc_rdm)
```

```
#> Iterations:
#>      preburn burn adapt sample
#> [1,]      0      0      0 1000
```

```

#> [2,]      0      0      0 1000
#> [3,]      0      0      0 1000
#>
#> mu
#>      B      v      v_lMd      t0
#> Rhat   1.001   1.004   1.002   1.001
#> ESS 2939.000 2120.000 2237.000 3016.000
#>
#> sigma2
#>      B      v      v_lMd      t0
#> Rhat   1.002   1.001   1.002   1.001
#> ESS 1864.000 1614.000 1670.000 1453.000
#>
#> alpha highest Rhat : 2
#>      B      v      v_lMd      t0
#> Rhat   1.003   1.004   1.003   1.002
#> ESS 2178.000 1872.000 1800.000 2723.000

```

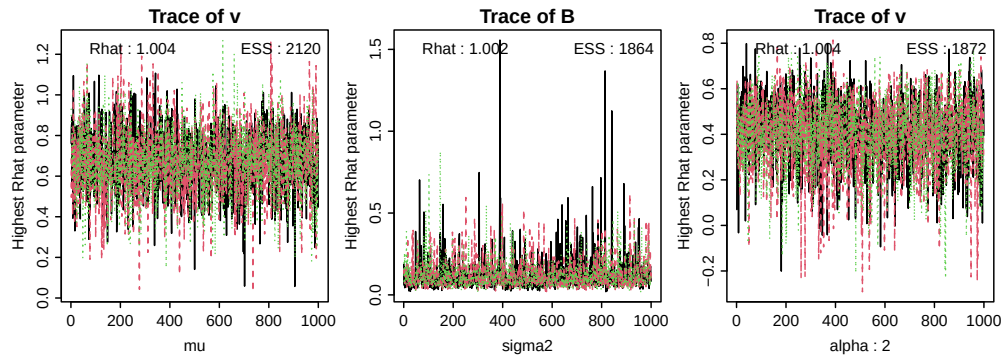


Figure 2: Trace plots of the sampled parameters with highest Rhat values (i.e., worst convergence) at the group-level mean (left, μ), the group-level variance (middle, σ^2), and the subject-level (α ; in this case, subject number 3). ESS = effective sample size.

We can plot the posterior distributions and obtain the credible intervals with `plot_pars()` and `credint()`, respectively. Both take an argument `selection` which can be used to specify which parameters should be plotted -- by default it is set to `mu`, corresponding to the group-level means, with alternative options `alpha` (subject-level), `sigma2` (group-level variance), `correlation` (group-level correlations).

```
plot_pars(emc_rdm)
```

By default, the `credint` function returns the credible intervals for the sampled parameters, but it can also return the credible intervals of implied posteriors after mapping and transforming:

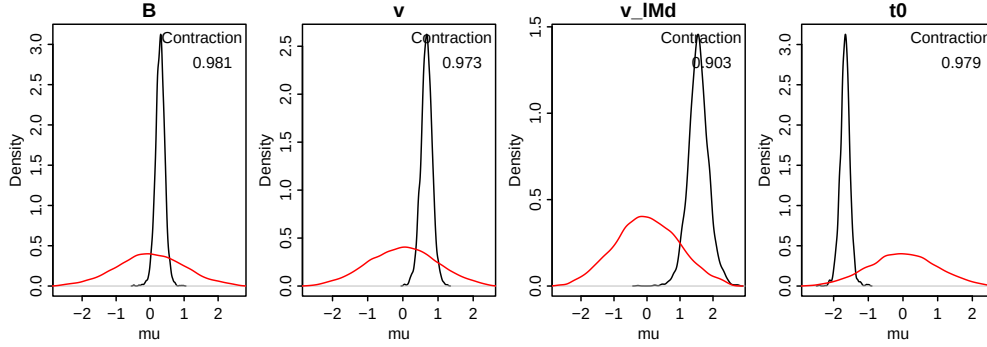


Figure 3: Posterior density (black) and prior density (red) of sampled group-level mean parameters of the baseline RDM. Contraction indicates the reduction of uncertainty in the posterior versus the prior (1 minus the ratio of the posterior to prior variance).

```
credint(emc_rdm)
#> $mu
#>      2.5%    50%  97.5%
#> B      0.016 0.298 0.568
#> v      0.343 0.673 0.998
#> v_lMd  0.957 1.553 2.194
#> t0     -1.962 -1.669 -1.405
credint(emc_rdm, map=TRUE)
#> $mu
#>      2.5%    50%  97.5%
#> v_lMFALSE 0.917 1.070 1.238
#> v_lMTRUE  4.209 4.323 4.443
#> B          1.340 1.404 1.475
#> t0          0.187 0.195 0.203
```

Once fit, we can generate posterior predictives with the `predict()` function, and plot the quality of fit with defective cumulative density function (CDF) plots:

```
pp_rdm <- predict(emc_rdm)
pp_wdm <- predict(emc_wdm)

plot_cdf(emc_rdm, pp_rdm,
  # with the factors argument, we request a separate CDF for each level of the factor S
  (stimulus)
  factors=c('S'))
```

```
plot_cdf(emc_wdm, pp_wdm, factors=c('S'))
```

Suggesting a slightly better fit of the RDM model, as seen in a better fit of the accuracy and 90th RT quantile. We can do formal model comparisons using DIC, BPIC, or marginal deviances using the `compare()` function:

```
compare(list(rdm=emc_rdm, wdm=emc_wdm))
#>      MD wMD    DIC wDIC  BPIC wBPIC EffectiveN meanD Dmean  minD
#> rdm -6576   1 -6654   1 -6600   1         54 -6707 -6733 -6761
#> wdm -5258   0 -5361   0 -5282   0         79 -5441 -5459 -5520
```

We can also use the posterior predictives to run a parameter recovery study. By default, `predict()` generates

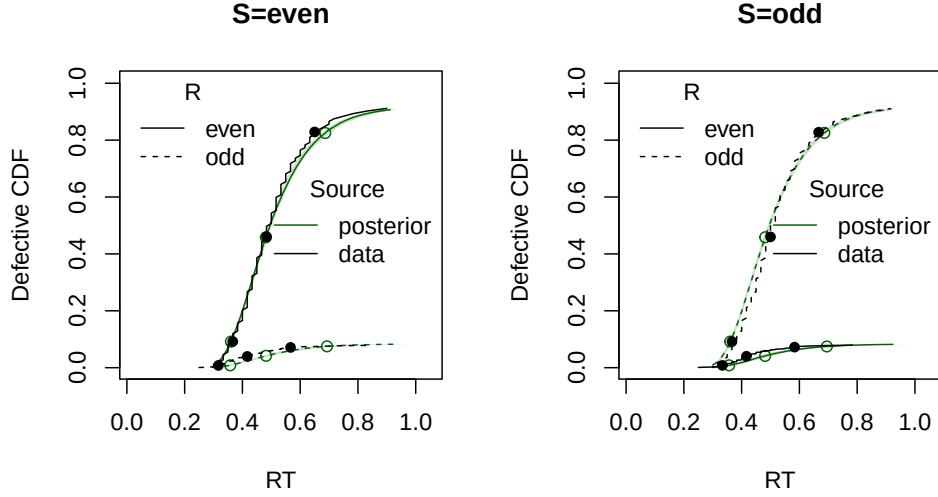


Figure 4: Defective cumulative density plots of the baseline RDM. Circles indicate the 10th, 50th, and 90ths quantiles of the response distributions for both response types R (solid = 'even', dotted = 'odd') in both conditions of the stimulus S (panels) for the data (black) and posterior predictives (green). The cumulative density functions are made defective as they sum to the proportion of each choice (rather than 1).

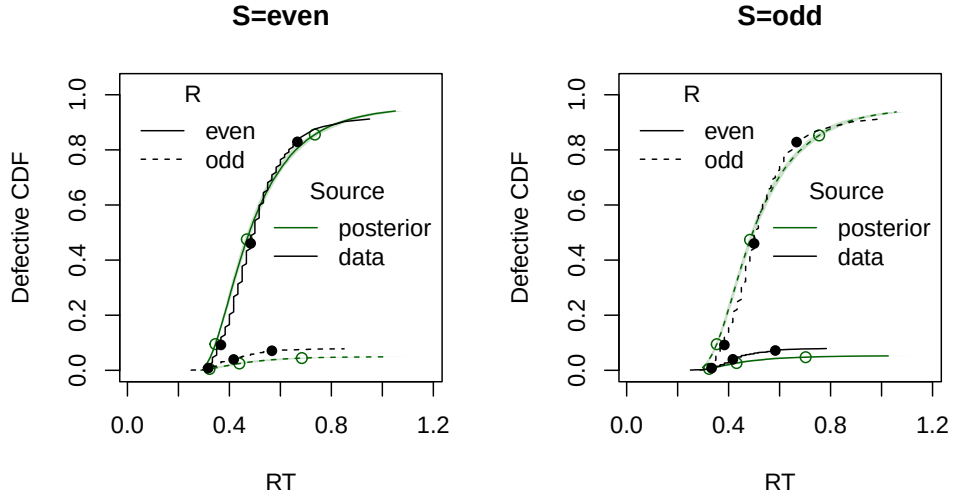


Figure 5: Defective cumulative density plots of the baseline WDM. Circles indicate the 10th, 50th, and 90ths quantiles of the response distributions for both response types R (solid = 'even', dotted = 'odd') in both conditions of the stimulus S (panels) for the data (black) and posterior predictives (green). The cumulative density functions are made defective as they sum to the proportion of each choice (rather than 1).

50 datasets (each marked with a unique number `postn`) and the data-generating parameters as an attribute. We can extract any of the generated datasets with the corresponding parameters, fit the same model again, and see whether the data-generating parameters correspond to the estimated parameters as follows:

```
postn <- 1
simulated_data <- pp_rdm[pp_rdm$postn == postn,]
true_pars <- attr(pp_rdm, 'pars')[[postn]]

emc_simulated_rdm <- make_emc(simulated_data, design_RDM)
emc_simulated_rdm <- fit(emc_simulated_rdm)
recovery(emc_simulated_rdm, true_pars, selection='alpha')
```

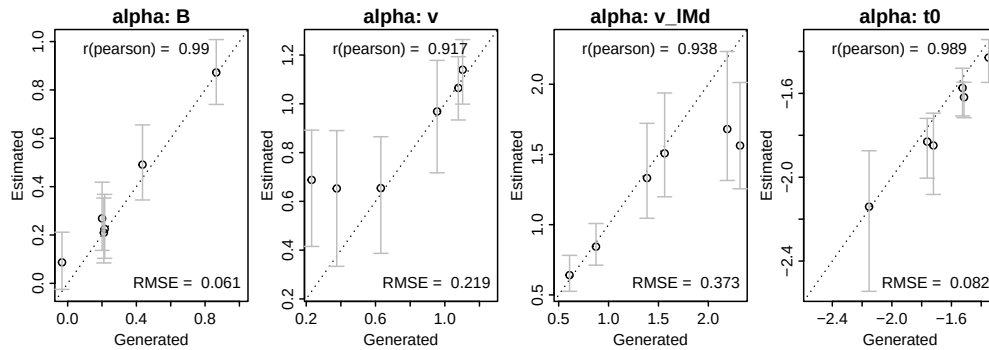


Figure 6: Parameter recovery results for the subject-level parameters. Estimated parameters (y-axis) are plotted against data-generating (x-axis). Error bars indicate 95% credible intervals, circles are on the median posterior, and diagonal lines is the identity. Pearson correlation between data-generating and posterior median. RMSE = root mean squared error.

EMC2 has built-in routines for simulation-based calibration that can be run on these models as well - for these, see `?run_sbc`.

Combined, this demonstrates some of the key elements of a principled workflow - including model specification, prior specification, fitting, convergence checks, posterior predictive checks, and validation. In the following, we turn to how to include time-varying parameters. In doing so, many of the above functions will be re-used. For the most part, this is a matter of additional detail in model specification, but there are important details in priors specifications that need to be adjusted, and additional useful plots, that we demonstrate below.

Trend specification

Both the descriptive trends and the formal mechanisms of dynamics work through `trend` objects in *EMC2*. In this object, you specify (1) the covariate of interest (e.g., time on task), (2) a kernel to apply to the covariate (e.g., linear, power, exponential, polynomial, or delta rule), and (3) which decision parameter is informed by the resulting covariate, and (4) the functional form of the mapping between the resulting covariate and the decision parameters, referred to as the 'base'. The `trend_help()` function gives an overview of the options:

```

trend_help()
#> Available kernels:
#>   custom: Custom C++ kernel: provided via register_trend().
#>   lin_decr: Decreasing linear kernel:  $k = -c$ 
#>   lin_incr: Increasing linear kernel:  $k = c$ 
#>   exp_decr: Decreasing exponential kernel:  $k = \exp(-d_{ed} * c)$ 
#>   exp_incr: Increasing exponential kernel:  $k = 1 - \exp(-d_{ei} * c)$ 
#>   pow_decr: Decreasing power kernel:  $k = (1 + c)^{-d_{pd}}$ 
#>   pow_incr: Increasing power kernel:  $k = 1 - (1 + c)^{-d_{pi}}$ 
#>   poly2: Quadratic polynomial:  $k = d1 * c + d2 * c^2$ 
#>   poly3: Cubic polynomial:  $k = d1 * c + d2 * c^2 + d3 * c^3$ 
#>   poly4: Quartic polynomial:  $k = d1 * c + d2 * c^2 + d3 * c^3 + d4 * c^4$ 
#>   delta: Standard delta rule kernel:  $k = q[i]$ .
#>           Updates  $q[i] = q[i-1] + \alpha * (c[i-1] - q[i-1])$ .
#>           Parameters:  $q0$  (initial value),  $\alpha$  (learning rate).
#>   delta2lr: Dual learning rate delta rule:  $k = q[i]$ .
#>           Like the standard delta rule, but with separate
#>           learning rates for positive and negative prediction errors.
#>           Parameters:  $q0$  (initial value),  $\alpha_{Pos}$  (learning rate for positive PEs),
#>            $\alpha_{Neg}$  (learning rate for negative PEs).
#>
#> Available base types:
#>   lin: Linear base:  $parameter + w * k$ 
#>   centered: Centered mapping:  $parameter + w * (k - 0.5)$ 
#>   add: Additive base:  $parameter + k$ 
#>   identity: Identity base:  $k$ 
#>
#> Phase options:
#>   premap: Trend is applied before parameter mapping. This means the trend parameters
#>           are mapped first, then used to transform cognitive model parameters before
#>           their mapping.
#>   pretransform: Trend is applied after parameter mapping but before transformations.
#>           Cognitive model parameters are mapped first, then trend is applied,
#>           followed by transformations.
#>   posttransform: Trend is applied after both mapping and transformations.
#>           Cognitive model parameters are mapped and transformed first,
#>           then trend is applied.

```

The last section, 'phase', warrants some extra attention. As demonstrated above, in *EMC2*, the user can define parameters using model formula language. *EMC2* uses these to create a design matrix by mapping the relevant factors to each design cell. In the case of the RDM, once mapped, in each design cell, only the v , B , $t0$, s , (and A , its omission in the design above causes it to be fixed to zero) parameters per accumulator remain. However, in many applications, the parameter of interest is defined as a between-accumulator difference (e.g., v_1Md in the RDM example above), or a between-condition difference (e.g., the effect of speed-accuracy trade-off cues on thresholds). These parameters only exist pre-mapping, and thus, the trend should be applied prior to mapping by setting `phase='premap'` (the default).

There is an important caveat here. By default, *EMC2* estimates some parameters with a lower bound (e.g., non-decision time, thresholds, RDM's drift rates), using a log scaling, and other parameters with both a lower and upper bound, using a probit (inverse cumulative normal) scaling. Parameters are *mapped* on the scale on which they are estimated, and *then* transformed. This also implies that if a trend is applied prior to mapping, the resulting parameters might later be transformed, leading to a potentially unwanted non-linear effect of the covariate on the parameter (see Figure 7).

Let's clarify with an example by trying to impose a linear trend on thresholds in case of the RDM:

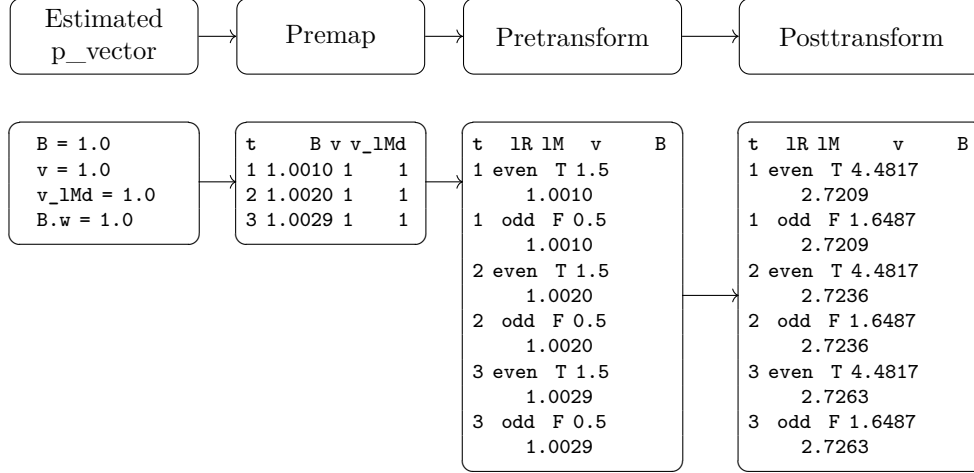


Figure 7: Illustration of parameter mapping and transformation in EMC2. The estimated parameter vector is first mapped, then transformed. In the illustration, only drift rates v and thresholds B are shown. In this example, the trend is applied **premap**, which leads the threshold to increase with trials (t). After mapping, the parameters are expanded to each design cell, with in this example two accumulators per trial (one for each latent response lR , and one matches the stimulus $lM=T$). Finally, the parameters are transformed to the natural scale. Since in this example, a linear trend is applied **premap**, this leads to an exponentially increasing trend **posttransform**.

```
# Add a 'trials' column specifying trial number
dat <- EMC2::add_trials(dat)

# Rescale the effect of this covariate to a larger parameter range to help
# sampling and interpretation (there are 1024 trials so the corresponding
# estimated parameter (B.w, see below) indexes the change from start to
# finish of the experiment).
dat$trials2 <- dat$trials/1024
lin_trend <- make_trend(cov_names='trials2', # covariate of interest
                      kernels='lin_incr',   # kernel
                      par_names='B',        # target parameter
                      bases='lin',          # functional form
                      phase='premap')        # and phase

RDM_lin_B <- design(model=RDM,
                   data=dat,
                   contrast=list(lM=ADmat),
                   matchfun=function(d) d$S==d$lR,
                   formula=list(B ~ 1, v ~ lM, t0 ~ 1),
                   covariates='trials2',   # specify relevant covariate columns
                   trend=lin_trend,         # add trend
                   report_p_vector=FALSE)  # suppress outputs for brevity
```

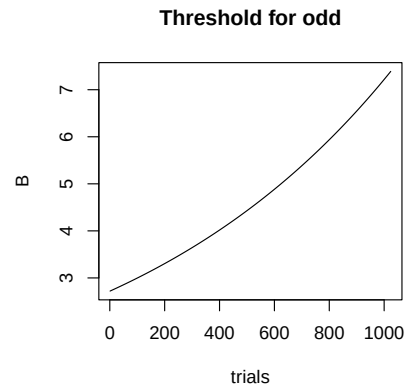
Note that we are now sampling one extra parameter compared to before, $B.w$, which is the weight of the influence of rescaled trials on thresholds: $B_t = B_0 + B.w * trials2$. Let's define a set of parameters and look at the resulting trial-by-trial thresholds:

```

emc_lin <- make_emc(dat, design=RDM_lin_B)
p_vector <- c('B'=1, 'v'=1, 'v_lMd'=1,
              't0'=0.1, 'B.w'=1)

# Visualize trend
plot_trend(p_vector, emc=emc_lin,
           par_name='B', subject=1,
           # filter here is a function that
           # selects only the rows in the dadm
           # corresponding to the 'odd' accumulator
           filter=function(data) data$lr=="odd",
           main='Threshold for odd')

```



Clearly, this is not a linear increase. This happens because the threshold parameter is sampled on the log scale, so an exponential transform is applied after mapping the parameter vector to the design cells. Since the linear trend was applied prior to mapping, this linear effect becomes non-linear. `mapped_pars()` can be used to clarify:

```

mapped_pars(RDM_lin_B)
#> $B
#>
#> intercept : exp(B_t)
#> Trends:
#> B_t = B + B.w * trials2
#>
#> $v
#> LM
#> TRUE : exp(v + 0.5 * v_lMd)
#> FALSE : exp(v - 0.5 * v_lMd)

```

There are two ways to solve this. One is to apply the trend after mapping and transforming, by setting the `phase` argument of `make_trend()` to `posttransform`. In this case, the sampler still estimates the target parameter on the log scale, but first transforms it back to the natural scale, and then adds the trend. While this will result in the expected linear trend (see example below), `posttransform` implies `postmap`, and many parameters of interest are defined only prior to mapping. So applying trends to those premap parameter types is not possible in combination with `posttransform`. Instead, the user can tell *EMC2* to estimate parameters on their natural scales by turning off transformations of the relevant parameters. Both approaches are exemplified below:

```

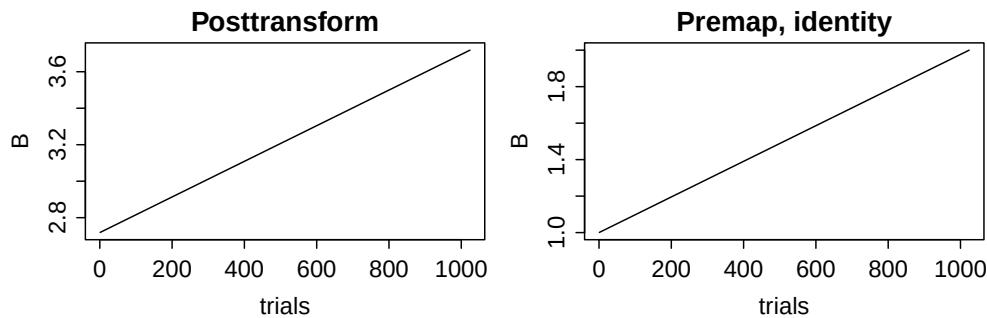
# Sample on log scale, but apply trend posttransform
lin_trend2 <- make_trend(cov_names='trials2', kernels='lin_incr',
                        par_names='B', bases='lin',
                        # We only change phase to posttransform:
                        phase='posttransform')
RDM_lin_B2 <- design(model=RDM,
                    data=dat,
                    contrast=list(lM=ADmat),
                    covariates='trials2',
                    matchfun=function(d) d$S==d$lR,
                    formula=list(B ~ 1, v ~ lM, t0 ~ 1),
                    trend=lin_trend2,
                    report_p_vector=FALSE)

# Alternatively, sample on natural scale, and apply trend premap
lin_trend3 <- make_trend(cov_names='trials2', kernels='lin_incr',
                        par_names='B', bases='lin',
                        phase='premap') # back to premap
RDM_lin_B3 <- design(model=RDM,
                    data=dat,
                    contrast=list(lM=ADmat),
                    covariates='trials2',
                    matchfun=function(d) d$S==d$lR,
                    # here, we tell EMC2 to sample the threshold on the natural scale
                    transform=list(func=c('B'='identity')),
                    formula=list(B ~ 1, v ~ lM, t0 ~ 1),
                    trend=lin_trend3,
                    report_p_vector=FALSE)

# Plot both trends with the same parameter vector
emc_lin2 <- make_emc(dat, design=RDM_lin_B2)
emc_lin3 <- make_emc(dat, design=RDM_lin_B3)
p_vector <- c('B'=1, 'v'=1, 'v_lMd'=1, 't0'=0.1, 'B.w'=1)

par(mfrow=c(1,2))
plot_trend(p_vector, emc=emc_lin2, par_name='B', subject=1,
          filter=function(data) data$lR=="odd",
          main='Posttransform')
plot_trend(p_vector, emc=emc_lin3, par_name='B', subject=1,
          filter=function(data) data$lR=="odd",
          main='Premap, identity')

```



Both lead to the expected linear effect. As a cautionary note, keep in mind that the default priors in *EMC2* are Gaussian distributions centered on 0 with unit variance. Since many cognitive model parameters cannot be negative (and by default *EMC2* will still enforce that, see help for the bound argument in `?design`), a

$N(0,1)$ prior is poorly chosen for those parameters that do not have support on the real line. For estimation and sampling, this usually has little influence in practice, but it may be important when estimating Bayes Factors.

With all that in mind, we can now start sampling our first model. We set a $N(2,0.5)$ prior on the threshold B to reduce the prior density on negative thresholds. This is a somewhat subjective choice based on earlier experience, so it reflects our (but perhaps not your) prior belief. The rest of the priors are left to their default $N(0,1)$ -- on the scale on which they are sampled.

```
prior_linB3 <- prior(RDM_lin_B3, mu_mean=c('B'=2), mu_sd=c('B'=0.5))
emc <- make_emc(dat, design=RDM_lin_B3, prior_list=prior_linB3)

# Priors on sampled parameters:
plot(prior_linB3, map=FALSE)
```

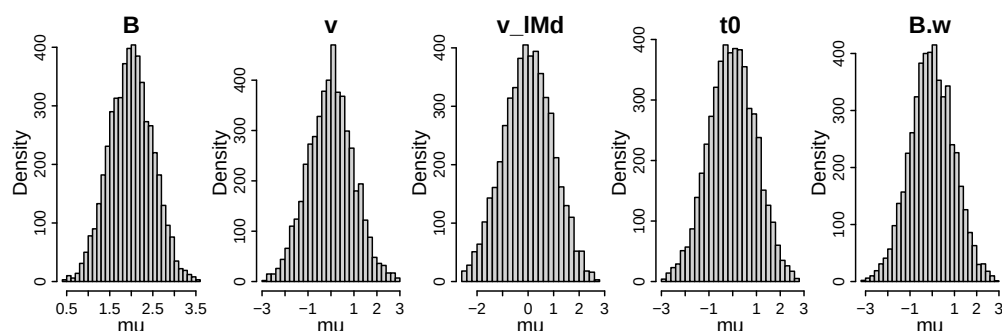


Figure 8: Prior distributions for the group-level mean parameters of the RDM with a linear trend, with the threshold B sampled in the natural scale.

We can now proceed with fitting, checking convergence, and plotting the posteriors and inspecting credible intervals:

```
emc <- fit(emc)
check(emc)
```

```
#> Iterations:
#>      preburn burn adapt sample
#> [1,]      0      0      0    1000
#> [2,]      0      0      0    1000
#> [3,]      0      0      0    1000
#>
#> mu
#>      B      v      v_lMd      t0      B.w
#> Rhat    1    1.001    1.001      1    1.001
#> ESS  2510 2450.000 2232.000 2349 2499.000
#>
#> sigma2
#>      B      v      v_lMd      t0      B.w
#> Rhat    1    1.002    1.001    1.006    1.004
#> ESS  1876 1576.000 1907.000 1247.000 1155.000
#>
#> alpha highest Rhat : 3
#>      B      v      v_lMd      t0      B.w
#> Rhat    1    1.003    1.002    1.001    1.007    1.002
#> ESS  1023.000 1362.000 1580.000 821.000 1343.000
```

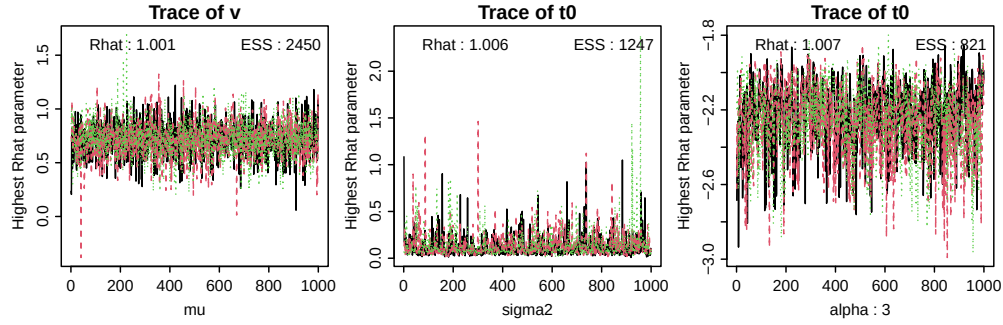


Figure 9: Trace plots of the sampled parameters with highest Rhat values (i.e., worst convergence) at the group-level mean (left, μ), the group-level variance (middle, σ^2), and the subject-level (α ; in this case, subject number 5). ESS = effective sample size.

```
plot_pars(emc)
```

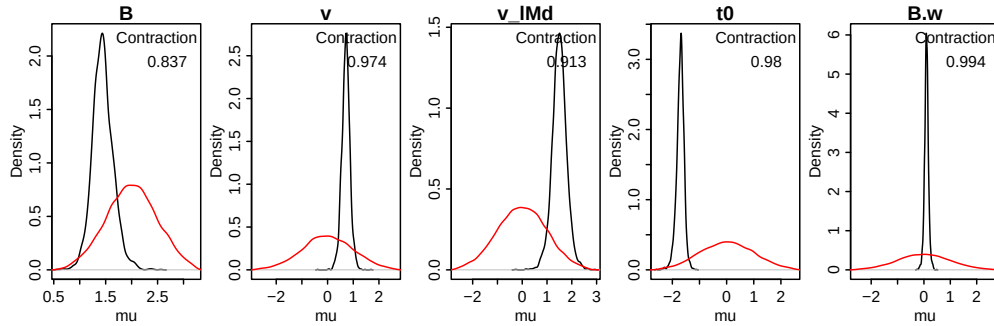


Figure 10: Posterior density (black) and prior density (red) of sampled group-level mean parameters of the RDM with a linear trend. Contraction indicates the reduction of uncertainty in the posterior versus the prior (1 minus the ratio of the posterior to prior variance).

```
credint(emc)
#> $mu
#>      2.5%    50%   97.5%
#> B      1.055  1.434  1.868
#> v      0.388  0.715  1.039
#> v_lMd  0.883  1.500  2.094
#> t0     -2.016 -1.696 -1.454
#> B.w     -0.063  0.097  0.256
```

On a group-level, we find a small (not credible) positive $B.w$ -- on average, in this dataset, thresholds appear to increase by ~ 0.099 over the course of 1024 trials. We can now generate posterior predictives, and while doing that, ask *EMC2* to return the mapped and transformed parameters, which will allow us to visualize the fitted thresholds over trials. Here we see that although there is little change for some participants, for others (e.g., 6) the change is more substantial.

```

# By setting return_trialwise_parameters to TRUE, we obtain the parameters
# of each iteration as an attribute in the returned object (pp).
pp <- predict(emc, return_trialwise_parameters=TRUE)
plot_trend(attr(pp, 'trialwise_parameters'), emc=emc,
           par_name='B',
           filter=function(data) data$1R == 'odd')

```

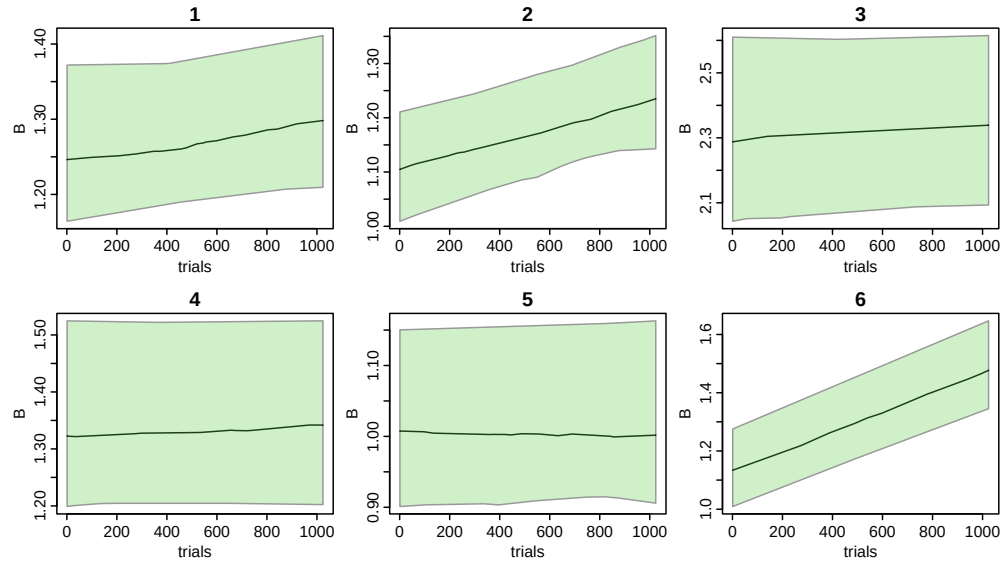


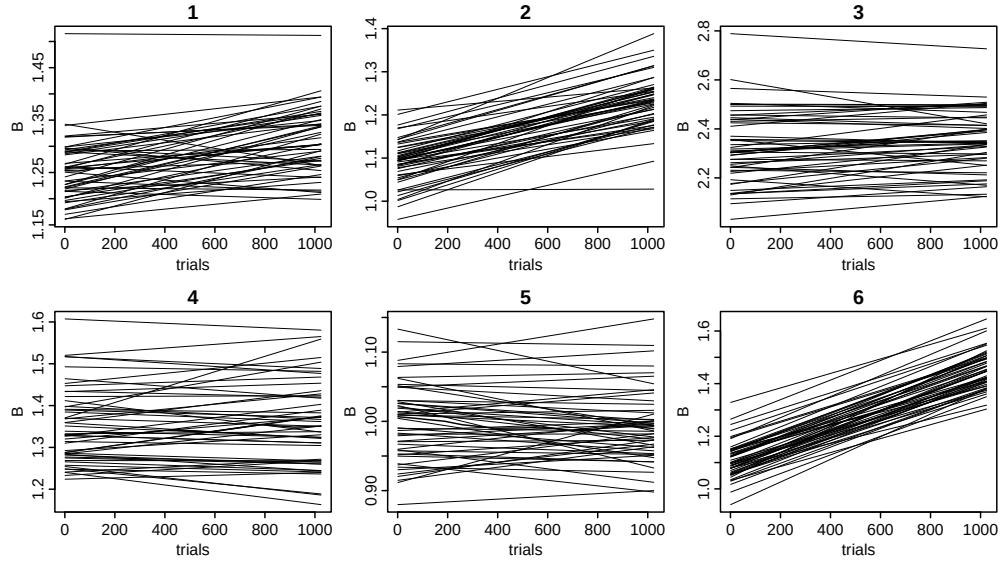
Figure 11: Estimated thresholds (B) as a function of trial number. Each panel corresponds to one participant (titles). Green shaded areas correspond to the 95% credible interval, the dark green line corresponds to the mean of the posterior predictives.

Here, the green shaded area indicated the 95% credible interval of the posterior on the threshold for each trial. We can also plot the individual thresholds of each posterior predictive separately as lines:

```

plot_trend(attr(pp, 'trialwise_parameters'), emc=emc,
           par_name='B',
           filter=function(data) data$1R == 'odd',
           pp_shaded=FALSE)

```

Exponential and power laws

Practice effects tend to be large initially and then gradually decay. Asymptotic functions like exponential or power functions can be used to model such effects. *EMC2* offers kernels that enforce either an increase or decrease. To demonstrate these functional forms:

```

trend_exp_incr <- make_trend(par_names='B', cov_names='trials2',
                             kernels='exp_incr', bases='lin')
trend_exp_decr <- make_trend(par_names='B', cov_names='trials2',
                              kernels='exp_decr', bases='lin')

# The two designs below are identical but for the trend argument
design_exp_incr <- design(model=RDM,
                          data=dat,
                          contrast=list(lM=ADmat),
                          covariates='trials2',
                          matchfun=function(d) d$S==d$lR,
                          transform=list(func=c('B'='identity')),
                          formula=list(B ~ 1, v ~ lM, t0 ~ 1),
                          trend=trend_exp_incr,
                          report_p_vector=FALSE)
design_exp_decr <- design(model=RDM,
                          data=dat,
                          contrast=list(lM=ADmat),
                          covariates='trials2',
                          matchfun=function(d) d$S==d$lR,
                          transform=list(func=c('B'='identity')),
                          formula=list(B ~ 1, v ~ lM, t0 ~ 1),
                          trend=trend_exp_decr,
                          report_p_vector=FALSE)

emc_incr <- make_emc(dat, design_exp_incr)
emc_decr <- make_emc(dat, design_exp_decr)

p_vector_incr <- c('B'=1, 'v'=1, 'v_lMd'=1, 't0'=0.1, 'B.w'=1, 'B.d_ei'=2)
p_vector_decr <- c('B'=1, 'v'=1, 'v_lMd'=1, 't0'=0.1, 'B.w'=1, 'B.d_ed'=2)

par(mfrow=c(1,2))
plot_trend(p_vector_incr, emc=emc_incr,
            par_name='B', subject=1, filter=function(data) data$lR == 'odd',
            main='B odd')
plot_trend(p_vector_decr, emc=emc_decr,
            par_name='B', subject=1, filter=function(data) data$lR == 'odd',
            main='B odd')

```

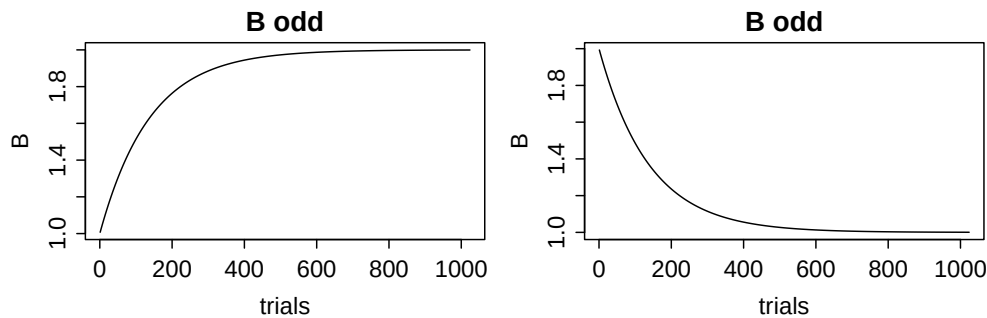


Figure 12: Example of increasing (top) and decreasing (bottom) exponential trends on threshold.

Note that the interpretation of the exponent ($B.e_{ei}$ or $B.e_{ed}$) depends on the direction: In the increasing case, it corresponds to the asymptote; in the decreasing case, to the intercept.

Advanced note: Enforcing a direction

Having both increasing and decreasing kernels may appear redundant, since the direction of the effect can also be flipped by the `w` parameter in the `lin` base. However, having both increasing and decreasing kernels facilitates enforcing a directional effect. For example, one might hypothesize that `v_lMd` (i.e., the ability to discriminate odd from even) increases sharply over the first few trials due to practice effects, and then stabilizes. An increase could be implemented with `exp_incr`, but if the sampler converges on negative values of `w` of the base, the resulting trend is actually decreasing rather than increasing.

We can force the sampler to sample only increasing trends by restricting the range of `w`. This can be done by sampling `w` on the log-scale, and transforming it to the natural scale. Note that transformations applied to parameters relating to the trends are always applied prior to estimating the trend.

```
trend_exp_incr <- make_trend(par_names='v_lMd', cov_names='trials2',
                             kernels='exp_incr', bases='lin')
design_exp_incr <- design(model=RDM,
                          data=dat,
                          contrast=list(LM=ADmat),
                          covariates='trials2',
                          matchfun=function(d) d$S==d$LR,
                          transform=list(func=c('v'='identity',
                                                  'v_lMd.w'='exp')),
                          formula=list(B ~ 1, v ~ LM, t0 ~ 1),
                          trend=trend_exp_incr,
                          report_p_vector=FALSE)
mapped_pars(design_exp_incr)
emc_incr <- make_emc(dat, design_exp_incr)
p_vector1 <- c('B'=1, 'v'=3, 'v_lMd'=1, 't0'=0.1, 'v_lMd.w'=-1, 'v_lMd.d_ei'=2)
p_vector2 <- c('B'=1, 'v'=3, 'v_lMd'=1, 't0'=0.1, 'v_lMd.w'=0, 'v_lMd.d_ei'=2)
p_vector3 <- c('B'=1, 'v'=3, 'v_lMd'=1, 't0'=0.1, 'v_lMd.w'=1, 'v_lMd.d_ei'=2)

par(mfcol=c(2,3))
plot_trend(p_vector1, emc=emc_incr, par_name='v',
            subject=1, filter=function(data) data$LM == TRUE,
            main='v matching', ylim=c(3.5,5))
plot_trend(p_vector1, emc=emc_incr, par_name='v',
            subject=1, filter=function(data) data$LM == FALSE,
            main='v mismatching', ylim=c(1, 2.5))
plot_trend(p_vector2, emc=emc_incr, par_name='v',
            subject=1, filter=function(data) data$LM == TRUE,
            main='v matching', ylim=c(3.5,5))
plot_trend(p_vector2, emc=emc_incr, par_name='v',
            subject=1, filter=function(data) data$LM == FALSE,
            main='v mismatching', ylim=c(1, 2.5))
plot_trend(p_vector3, emc=emc_incr, par_name='v',
            subject=1, filter=function(data) data$LM == TRUE,
            main='v matching', ylim=c(3.5,5))
plot_trend(p_vector3, emc=emc_incr, par_name='v',
            subject=1, filter=function(data) data$LM == FALSE,
            main='v mismatching', ylim=c(1, 2.5))
```

```
#> $v
#> LM
#> TRUE : v + 0.5 * v_lMd_t
#> FALSE : v - 0.5 * v_lMd_t
#> Trends:
#> v_lMd_t = v_lMd + exp(v_lMd.w) * (1 - exp(-exp(v_lMd.d_ei) * trials2))
```

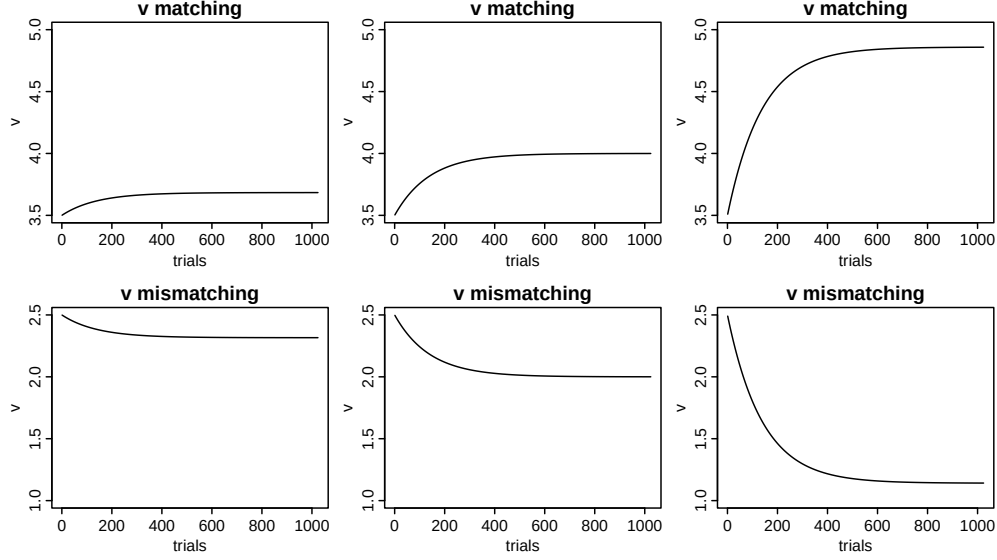


Figure 13: Demonstration of enforcing a direction of an effect. Irrespective of the sign of $v_1Md.w$ (-1 for column 1, 0 for column 2, and 1 for column 3), the difference between the matching and mismatching accumulator's drift rate increases over trials when $v_1Md.w$ is estimated on the log scale and the increasing exponential kernel is used.

In this set-up, no matter the value of $v_1Md.w$ the between-accumulator difference in drift rates increases over time. Using the same logic, we can also enforce a decrease by using `exp_decr` as a kernel and applying an exponential `transform` to w .

So far, we only inspected time on task as defined by trial number. Other options include time on task within block (e.g., short breaks between trials could cause an increased threshold for the first few trials in each block), or the number of times a stimulus type has been shown. Furthermore, the parameters describing the trend could differ between blocks and stimulus types. To test such hypotheses, we can allow the trend parameters themselves to vary with experimental factors, as defined in `formula` in `design()`. The example data does not have stimulus type recorded, and all trials were run in a single block, but for demonstrative purposes we can simulate such effects:

```

# simulate block number (assume 3 blocks)
dat$block <- as.factor(as.numeric(cut(dat$trials, breaks=3)))
for(subject in unique(dat$subjects)) {
  dat[dat$subjects==subject, 'trial_in_block'] <- ave(
    seq_along(dat[dat$subjects==subject, 'block']),
    dat[dat$subjects==subject, 'block'], FUN=seq_along)/1024
}
# simulate presented digit
dat$stimulus_repetition <- ave(seq_along(dat$S),
                              dat$S, FUN=seq_along)

# combine two trends
trends <- make_trend(par_names=c('B', 'v_lMd'),
                    cov_names=c('trial_in_block', 'stimulus_repetition'),
                    kernels=c('exp_decr', 'exp_incr'),
                    bases=c('lin', 'lin'))
design_multitrend <- design(model=RDM,
                          data=dat,
                          contrast=list(LM=ADmat),
                          covariates=c('trial_in_block',
                                       'stimulus_repetition'),
                          matchfun=function(d) d$S==d$LR,
                          transform=list(func=c('v_lMd.w'='exp',
                                                  'B.w'='exp',
                                                  'v'='identity',
                                                  'B'='identity')),
                          formula=list(B ~ 1, v ~ LM, t0 ~ 1, `B.d_ed`~block),
                          trend=trends,
                          report_p_vector=FALSE)
emc_multitrend <- make_emc(dat, design_multitrend)

# thresholds
p_vector <- c('B'=1, 'v'=1, 'v_lMd'=2, 't0'=0.1,
              'B.d_ed'=2, 'B.d_ed_block2'=1, 'B.d_ed_block3'=1, 'B.w'=.25,
              'v_lMd.w'=1, 'v_lMd.d_ei'=.2)

par(mfcol=c(1,3))
plot_trend(p_vector, emc=emc_multitrend, par_name='B',
           subject=1, filter=function(data) data$LR == 'odd',
           main='Thresholds', xlab='Trial number')
plot_trend(p_vector, emc=emc_multitrend, par_name='v',
           subject=1, filter=function(data) data$LM == TRUE,
           main='Drift rate (matching)', on_x_axis='stimulus_repetition',
           xlab='Stimulus repetition', xlim=c(1,20))
plot_trend(p_vector, emc=emc_multitrend, par_name='v',
           subject=1, filter=function(data) data$LM == TRUE,
           main='Drift rate (matching)', xlab='Trial number', xlim=c(1,20))
mapped_pars(design_multitrend)

```

```

#> $B
#>
#> intercept : B_t
#> Trends:
#> B_t = B + exp(B.w) * exp(-exp(B.d_ed) * trial_in_block)
#>
#> $v
#> LM
#> TRUE : v + 0.5 * v_lMd_t

```

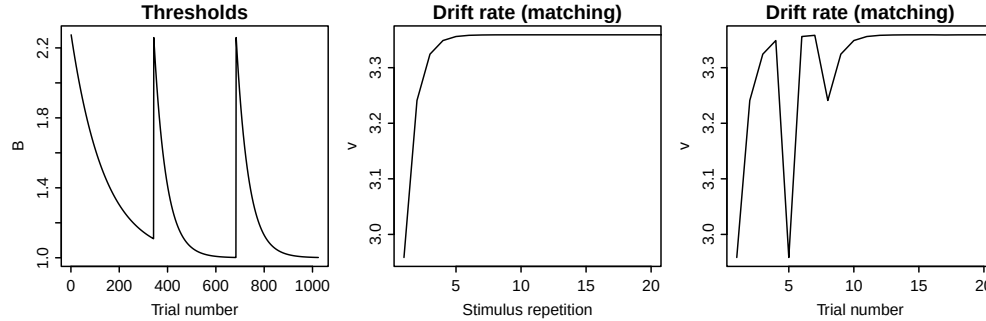


Figure 14: Effect of specifying multiple trends with parameters that vary between blocks (left) or conditions (middle, right).

```
#> FALSE : v - 0.5 * v_lMd_t
#> Trends:
#> v_lMd_t = v_lMd + exp(v_lMd.w) * (1 - exp(-exp(v_lMd.d_ei) * stimulus_repetition))
#>
#> $B.d_ed
#> block
#> 1 : exp(B.d_ed)
#> 2 : exp(B.d_ed + B.d_ed_block2)
#> 3 : exp(B.d_ed + B.d_ed_block3)
```

The left panel illustrates how the thresholds decrease within each block (at different rates between blocks), but then reset to the same intercept after every break. The middle panel shows the drift rate for the correct accumulator as a function of how often a stimulus has been presented. Note that, when plotted against trial number (right panel), this leads to strong variability in rates across trials.

Polynomials

When a researcher has no strong prior assumptions about the shape of the trend but wishes to capture gradual changes, it is possible to fit a set of basis functions, such as polynomials. *EMC2* currently includes second-, third-, and fourth-order polynomials as available kernels. Many functions can be approximated with increasing accuracy by summing polynomials of increasing order. However, the increased accuracy comes at a cost of increased sampling uncertainty, since more parameters need to be estimated. As such, it may be possible to estimate only a few polynomials.

```
trend_help(kernel='poly4')
#> Description:
#> Quartic polynomial: k = d1 * c + d2 * c^2 + d3 * c^3 + d4 * c^4
#>
#> Default transformations (in order):
#> list(d1 = "identity", d2 = "identity", d3 = "identity", d4 = "identity")
#>
#> Available bases, first is the default:
#> add, identity
#>
```

Note that the kernel parameters (*d1*, *d2*, *d3*, *d4*) already weigh the contributions of the individual polynomial terms. As such, this kernel should not be combined with a linear base (*base='lin'*), but instead with an additive base (*base='add'*). Additionally, we should center the polynomial basis around the mean of the covariate (allowing there to be changes in the direction of the trend, which is not possible if the covariate is

always positive). The following demonstrates the flexibility of a 4th order polynomial with a few randomly chosen parameter sets:

```
dat$tctrd <- dat$trials2-mean(dat$trials2)
trend_poly <- make_trend(par_names='B',
                        cov_names='tctrd',
                        kernels='poly4',
                        bases='add')
design_poly <- design(model=RDM,
                    data=dat,
                    contrast=list(lM=ADmat),
                    covariates='tctrd',
                    matchfun=function(d) d$S==d$lR,
                    transform=list(func=c('B'='identity')),
                    formula=list(B ~ 1, v ~ lM, t0 ~ 1),
                    trend=trend_poly,
                    report_p_vector=FALSE)
mapped_pars(design_poly)
emc_poly <- make_emc(dat, design_poly)

p_vector1 <- c('B'=1, 'v'=1, 'v_lMd'=2, 't0'=0.1,
               'B.d1'=2, 'B.d2'=3, 'B.d3'=-5, 'B.d4'=-5)
p_vector2 <- c('B'=1, 'v'=1, 'v_lMd'=2, 't0'=0.1,
               'B.d1'=-1, 'B.d2'=0, 'B.d3'=-5, 'B.d4'=0)
p_vector3 <- c('B'=1, 'v'=1, 'v_lMd'=2, 't0'=0.1,
               'B.d1'=0, 'B.d2'=2, 'B.d3'=8, 'B.d4'=8)
p_vector4 <- c('B'=1, 'v'=1, 'v_lMd'=2, 't0'=0.1,
               'B.d1'=1, 'B.d2'=-3, 'B.d3'=-2, 'B.d4'=10)

par(mfrow=c(1,4))
plot_trend(p_vector1, emc=emc_poly, par_name='B',
           subject=1, filter=function(data) data$lR == 'odd',
           main='p_vector1', xlab='Trial number')
plot_trend(p_vector2, emc=emc_poly, par_name='B',
           subject=1, filter=function(data) data$lR == 'odd',
           main='p_vector2', xlab='Trial number')
plot_trend(p_vector3, emc=emc_poly, par_name='B',
           subject=1, filter=function(data) data$lR == 'odd',
           main='p_vector3', xlab='Trial number')
plot_trend(p_vector4, emc=emc_poly, par_name='B',
           subject=1, filter=function(data) data$lR == 'odd',
           main='p_vector4', xlab='Trial number')
```

```
#> $B
#>
#> intercept : B_t
#> Trends:
#> B_t = B + (B.d1 * tctrd + B.d2 * tctrd^2 + B.d3 * tctrd^3 + B.d4 * tctrd^4)
#>
#> $v
#> lM
#> TRUE : exp(v + 0.5 * v_lMd)
#> FALSE : exp(v - 0.5 * v_lMd)
```

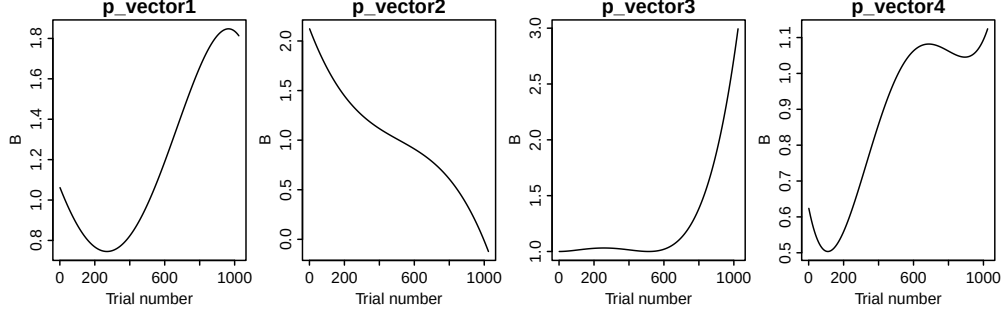


Figure 15: Illustration of the flexibility of the polynomial kernel: With four different (arbitrarily chosen) parameter sets, widely different shapes of trends can be obtained. Here, they are visualised as trends on thresholds B , but they can be on any parameter.

Mechanisms of dynamics: Stimulus memory

Models with dynamic mechanisms (as opposed to the descriptions of dynamics in the last section) often rely on delta rules. To implement delta rules, we follow the same general logic as above --- the delta rule is implemented as a kernel in `make_trend`. Let's implement the stimulus memory mechanism proposed by Miletić et al. (2025). Here, the decision maker keeps track of the probability that a stimulus is even (or odd). The covariate in this case is the stimulus, which we recode as 1 (even) and -1 (odd) and use in the delta rule:

$$Q_{SM,t+1} = Q_{SM,t} + \alpha_{SM} \cdot (S_t - Q_{SM,t}), \quad S \in \{-1, 1\}$$

The delta rule introduces two extra parameters: a learning rate α_{SM} , and a value for Q at the start of the experiment $Q_{SM,0}$. The latter is hard to estimate, and we tend to fix it to a constant—for this rule, a constant of 0 implies a belief that both stimuli occur equally often.

For now, let us assume that stimulus memory influences the relative thresholds: the threshold for the even accumulator increases, and the odd threshold decreases (by the same amount). To achieve this, we need to parametrise thresholds with the same mean-difference parametrisation as we used for drift rates. That is,

$$\begin{aligned} B_{odd} &= B_{mean} + B_{lRd} \\ B_{even} &= B_{mean} - B_{lRd} \end{aligned}$$

The B_{lRd} term is the parameter influenced by the updated covariate --- however, since we do not necessarily expect an across-trial *mean* difference between thresholds, we set B_{lRd} as a constant to 0.

Using a linear base, we then allow the threshold difference to vary on a trial-by-trial basis with

$$B_{lRd,t} = B_{lRd} + w_{SM} \cdot Q_{SM,t}$$

Note the extra parameter here: w_{SM} .


```

dat$Stim1 <- ifelse(dat$S=='even', 1, -1)
SM_trend <- make_trend(cov_names=c('Stim1'),
                      kernels='delta',
                      par_names='B_lRd',
                      bases='lin',
                      phase="premap")
RDM_SM <- design(model=RDM,
                 data=dat,
                 contrast=list(LM=ADmat, LR=ADmat),
                 covariates='Stim1',
                 matchfun=function(d) d$S==d$LR,
                 formula=list(B ~ LR, v ~ LM, t0 ~ 1),
                 trend=SM_trend,
                 constants=c('B_lRd'=0, 'B_lRd.q0'=0),
                 report_p_vector=FALSE)

```

We can now apply a similar prior to B as above, and fit the model. Note that data compression is turned off automatically, since the delta rule needs to be applied to the covariate on each trial individually.

```

prior_SMB <- prior(RDM_SM, mu_mean=c(B=2), mu_sd=c(B=0.5))
emc <- make_emc(dat, design=RDM_SM, prior_list=prior_SMB)
emc <- fit(emc)
check(emc)

```

```

#> Iterations:
#>      preburn burn adapt sample
#> [1,]      0      0      0    1000
#> [2,]      0      0      0    1000
#> [3,]      0      0      0    1000
#>
#> mu
#>      B      v      v_lMd      t0 B_lRd.w B_lRd.alpha
#> Rhat   1.016   1.006   1.003   1.006   1.014   1.007
#> ESS 1085.000 1682.000 1568.000 1559.000 974.000   665.000
#>
#> sigma2
#>      B      v      v_lMd      t0 B_lRd.w B_lRd.alpha
#> Rhat   1.017   1.006   1.005   1.006   1.023   1.101
#> ESS 1004.000 1212.000 1231.000 1318.000 732.000   305.000
#>
#> alpha highest Rhat : 5
#>      B      v      v_lMd      t0 B_lRd.w B_lRd.alpha
#> Rhat   1.005   1.009   1.009   1.006   1.015   1.037
#> ESS 1202.000 1001.000 1067.000 1371.000 360.000   213.000

```

```

plot_pars(emc)

```

We find excellent convergence properties, especially given learning rate parameters are often hard to estimate. Note that learning rates are by default sampled on the probit scale, so if we want to know the mean learning rate, we need to map and transform the parameters first (this can take a while to run):

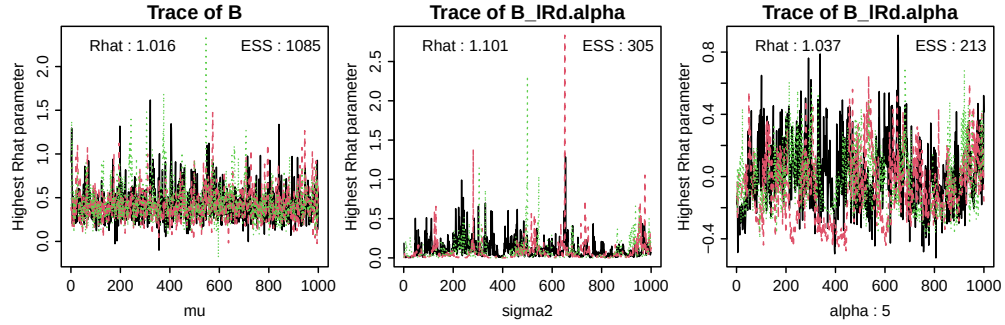


Figure 16: Trace plots of the sampled parameters with highest Rhat values (i.e., worst convergence) at the group-level mean (left, μ), the group-level variance (middle, σ^2), and the subject-level (α ; in this case, subject number 5). ESS = effective sample size.

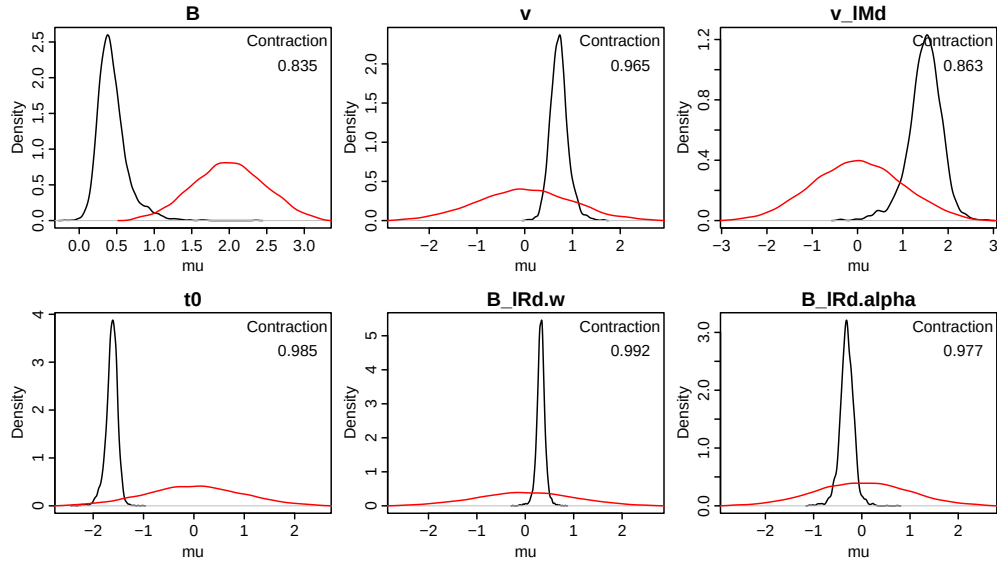


Figure 17: Posterior density (black) and prior density (red) of sampled group-level mean parameters of the RDM with an SM mechanism. Contraction indicates the reduction of uncertainty in the posterior versus the prior (1 minus the ratio of the posterior to prior variance).

```

credint(emc, map=TRUE)
#> $mu
#>          2.5%  50% 97.5%
#> v_lMFALSE 0.848 0.988 1.130
#> v_lMTRUE  4.252 4.363 4.475
#> B_lReven  1.314 1.367 1.426
#> B_lRodd   1.314 1.367 1.426
#> t0        0.208 0.215 0.222
#> B_lRd.w   0.274 0.321 0.366
#> B_lRd.alpha 0.336 0.392 0.462

```

Hence, we find a group-wise mean learning rate of ~ 0.39 , and the threshold of the odd accumulator is 0.324 lower than the threshold for the even accumulator if the participant has a Q_{SM} -value of 1.

We can now check whether the SM mechanism can explain stimulus-history effects by plotting the RTs (by choice) and the probability of choosing left as a function of the previous three presented stimuli. First, generate posterior predictives, and then plot the stimulus history effects:

```

pp <- predict(emc, return_trialwise_parameters=TRUE)
# this is a convenience function that is not part of EMC2. For help, inspect the
# first lines after calling plot_history_effects (omitting the parentheses),
# which document its use
plot_history_effects(dat, pp, n_hist=3)

```

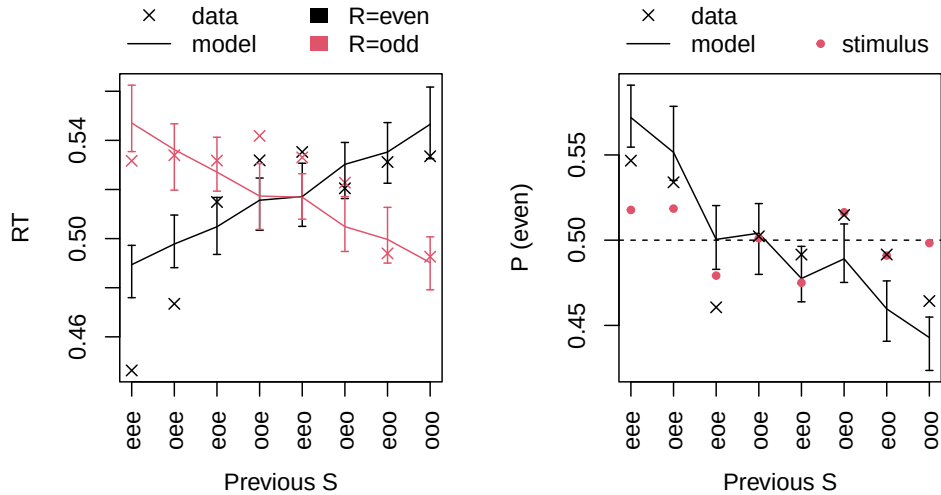


Figure 18: Effects of local stimulus (S) history on response times (left) and response (R) probability (right). X-axis labels indicate the three most recent trials' stimuli (e.g., eee = even, even even, eeo = even, even, odd; the right-most letter in the string indicates the most recent trial). Crosses indicate data, model predictions are indicated by error bars (95% credible intervals), and lines connect medians. Red circles in the right plot indicate the probability of the stimulus, which is an important quality check.

The left panel of this plot shows the mean RT as a function of the three previous trials' stimuli (e.g., ooo = odd, odd, odd; ooe = odd, odd, even; etc, where the right-most letter in the string indicates the most recent trial). The RT data show a cross-over pattern, such that if the previous three stimuli were odd, 'odd' responses are faster than 'even' responses; and vice versa. The model tends to capture this cross-over pattern fairly well, with some misfits mostly due to an apparent asymmetry in the cross-over in the data.

The right panel shows the probability of choosing even (black) and the probability of the stimulus being even (red) as a function of local trial history. The black crosses demonstrate that participants are more likely ($\sim .55$) to choose 'even' if the previous three trials were 'even'. The red circles are important as a quality check. If the stimuli are generated truly randomly, the red circles should be approximately at 0.5 in all cases. However, some experiments use pseudorandomization, which can lead to negative correlations in local stimulus histories. This can confound any stimulus history effects present in the human data. True stimulus history effects are those that are larger than the ones present in the data, e.g., visible as black crosses deviating more from 0.5 compared to red circles.

Advanced note: at

Under the hood, EMC2 works with data-augmented design matrices (**dadms**). A **dadm** contains one row per accumulator per trial – so for race models applied to two-alternative forced choice tasks, this will typically result in two rows per trial. We can inspect the **dadm** for the first subject in the **emc** object to see that the covariate **Stim1** is indeed the same across accumulators in each trial:

```
head(emc_rdm[[1]]$data[[1]])
#>  subjects      S      R      rt trials      lR      lM winner
#> 1      1 even even 0.5333333      1 even  TRUE   TRUE
#> 2      1 even even 0.5333333      1 odd  FALSE  FALSE
#> 3      1 even even 0.4166667      2 even  TRUE   TRUE
#> 4      1 even even 0.4166667      2 odd  FALSE  FALSE
#> 5      1 even even 0.5000000      3 even  TRUE   TRUE
#> 6      1 even even 0.5000000      3 odd  FALSE  FALSE
```

The delta rule is applied to the covariate as it is specified in the **dadm**. Since, in the examples in this tutorial, the covariate is the same for both accumulators, applying a delta rule to the covariate in the **dadm** would lead to *two* updates every trial: One for the first accumulator, and then another one for the second accumulator. The Q-values would then also differ between accumulators. The correct behavior is to update only once every trial (i.e., for the first accumulator), and then feed forward the updated Q-value to every second (and third, fourth, ...) accumulator. By setting 'at='lR'', the Q-value is only updated at the first level of the factor in 'lR', and the resulting Q-value is carried over to the other levels of 'lR'.

In many applications when using dynamic EAMs, this is likely the intended behavior, and therefore 'lR' is the default setting for 'at'. The DDM marks an exception, since it assumes one accumulator per trial, so has no 'lR' factor, as we demonstrate below. Additionally, advanced use cases might need to set 'at=NULL'.

As before, we can plot the trial-wise parameters with `plot_trend()`. Let's zoom in on the even and odd thresholds of the first three subject for the first 20 trials:

```
par(mfcol=c(2,3))
for(subject in 1:3) {
  plot_trend(attr(pp, 'trialwise_parameters'), emc=emc,
             par_name='B', subject=subject,
             filter= function(data) data$lR == 'odd',
             main=paste0(subject, ' odd'), xlim=c(1,20))
  plot_trend(attr(pp, 'trialwise_parameters'), emc=emc,
             par_name='B', subject=subject,
             filter= function(data) data$lR == 'even',
             main=paste0(subject, ' even'), xlim=c(1,20))
}
```

Alternatively, we can plot the evolution of the Q-values themselves. Note that `predict()` was run with

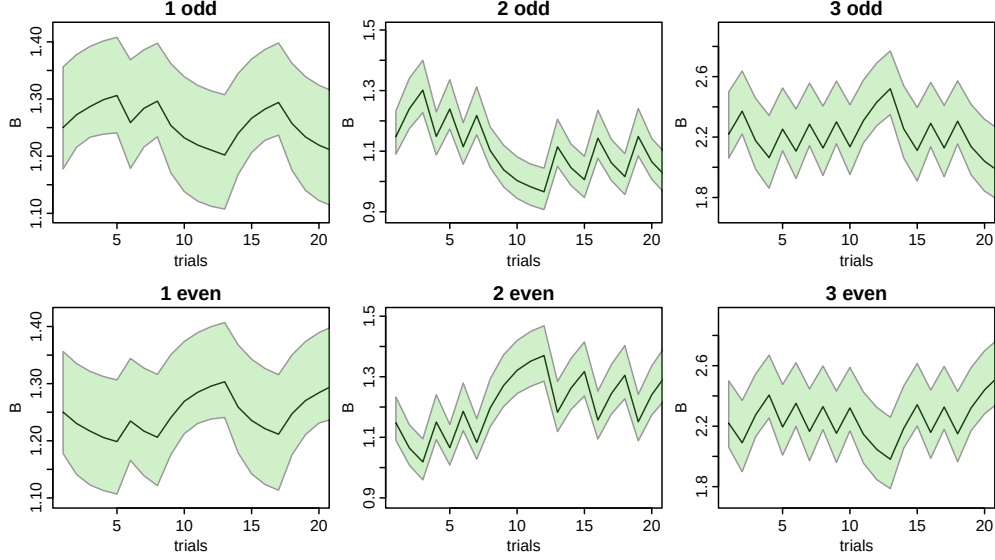


Figure 19: Estimated trialwise thresholds of the accumulator corresponding to responding odd (top) and even (bottom) for the first three participants (columns). Green shaded areas indicate the 95% credible interval.

another argument: `return_trialwise_parameters`, which returns the updated covariates as an attribute. We can use this as follows:

```
plot_trend(attr(pp, 'trialwise_parameters'), emc=emc,
           par_name='B_lRd.Stim1.Qvalue',
           xlim=c(1,20))
```

Repetition/alternation memory

The stimulus-memory mechanism accounts for what (Jones et al. 2013) termed *first-degree* recency effects: They depend on stimulus identity (e.g., 'odd' or 'even'). There are also *second-degree* recency effects, which depend on sequential properties - i.e., whether stimuli are repetitions or alternations of those stimuli before. It is easy to implement such a mechanism with a delta rule as well. If we code the stimuli as $S_t \in \{-1, 1\}$, then the multiplication $S_{t-1}S_t$ indicates if trial t repeats (outcome 1) or alternates (outcome -1) trial $t-1$. Let's hypothesize that participants keep track of the repetition rate using a repetition memory (RM). That could make use of the delta rule:

$$Q_{RM,t+1} = Q_{RM,t} + \alpha_{RM} \cdot (S_{t-1}S_t - Q_{RM,t}), \quad S \in \{-1, 1\}$$

Q_{RM} is a latent representation for the belief that the stimulus will repeat itself. We could hypothesize that, if participants believe that the stimulus is repeating, the threshold corresponding the choice option that repeats the previous stimulus is lower compared to the one that alternates - and vice versa. To implement this, we need to parametrise the thresholds as repeat/alternate:

$$\begin{aligned} B_{repeat} &= B_{mean} + B_{lR2d} \\ B_{alternate} &= B_{mean} - B_{lR2d} \end{aligned}$$

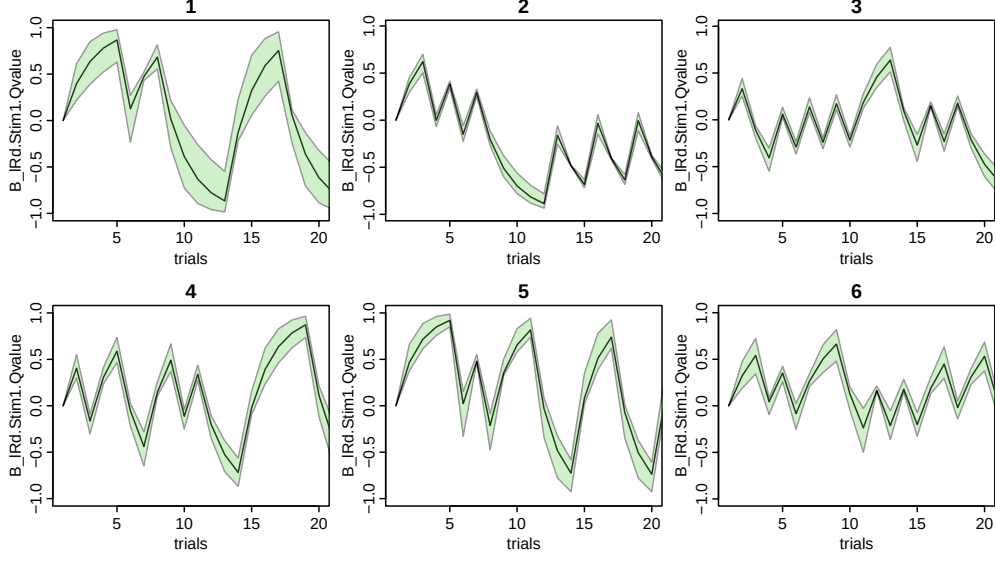


Figure 20: Estimated Q-values corresponding to the SM mechanism for all six participants participants (panels). Green shaded areas indicate the 95% credible interval.

Like before, we are making use of an advantage-difference parametrisation for the thresholds here, but the difference B_{lR2d} parameter here codes accumulators by whether they repeat or alternate the previous stimuli. This B_{lR2d} term is the parameter influenced by the updated covariate. Since we do not necessarily expect an across-trial *mean* difference between thresholds, we set B_{lR2d} itself as a constant to 0.

Using a linear base, we then allow the threshold difference to vary on a trial-by-trial basis with

$$B_{lR2d,t} = B_{lR2d} + w_{RM} \cdot Q_{RM,t}$$

Again introducing a weighting parameter: w_{RM} .

Let's implement this step by step. First, let's add $S_{t-1}S_t$ to the data, as `dat$Stim2`. This will be our new covariate. It should affect `B_1R2d` with a linear basis. Contrary to `1R`, the `1R2` factor is not automatically created by *EMC2*, so we must pass a function to design that creates it for us.

```

# function to lag
lag <- function(x, shift = 1) {
  n <- length(x)
  if (shift == 0) return(x)

  ret <- rep(NA, n)
  if (abs(shift) < n) {
    if (shift > 0) {
      ret[-seq_len(shift)] <- x[seq_len(n - shift)]
    } else {
      ret[seq_len(n + shift)] <- x[(1 - shift):n]
    }
  }
}

attributes(ret) <- attributes(x)
ret
}

dat$Stim1 <- ifelse(dat$S=='even', 1, -1) # S_{t}
dat$Stim2 <- dat$Stim1*lag(dat$Stim1,1) # S_{t-1}S_{t}
RM_trend <- make_trend(cov_names=c('Stim2'),
                      kernels='delta',
                      par_names='B_LR2d',
                      bases='lin',
                      phase="premap")
ADmat <- matrix(c(-.5,.5), ncol=1, dimnames=list(NULL,'d'))
make_LR2 <- function(x) {
  # note that we need to use lag 2, since the dadm has one row per accumulator
  LR2 <- as.numeric(x$LR)==as.numeric(lag(x$S,2))
  # The first trial has no repeat/alternate.
  # However, since we set B_LR2d.q0, we can set LR2 for this trial
  # arbitrarily TRUE or FALSE -- B_LR2d_{1} is always 0.
  LR2[is.na(LR2)] <- TRUE
  factor(LR2, levels=c(1,0), labels=c('rep', 'alt'))
}

RDM_RM <- design(model=RDM,
                 data=dat,
                 contrast=list(lM=ADmat, lR=ADmat, lR2=ADmat),
                 covariates='Stim2',
                 functions=list(LR2=make_LR2),
                 matchfun=function(d) d$S==d$LR,
                 formula=list(B ~ LR2, v ~ lM, t0 ~ 1),
                 trend=RM_trend,
                 constants=c('B_LR2d'=0, 'B_LR2d.q0'=0),
                 report_p_vector=FALSE)

```

Before fitting, we might want to simulate some data to test whether simulated data indeed demonstrates second-degree recency effects.

```

p_vector <- sampled_pars(RDM_RM)
p_vector[1:6] <- c(log(2), log(1), .5, log(.15), 2, qnorm(0.1))
simDat <- make_data(p_vector, RDM_RM, data=dat)
# the plot_history_effects()-function takes an additional argument:
# degree, which can be 1 for first-degree or 2 for second-degree
# effects
plot_history_effects(dat=simDat, degree=2)

```

Which shows the expected pattern: if the previous trials were repeats, the model predicts participants are

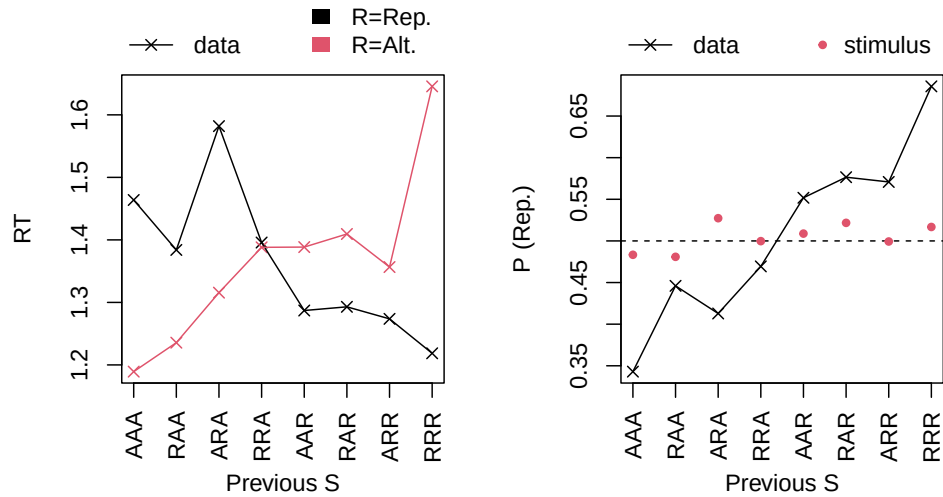


Figure 21: Effects of local stimulus (S) repetition history on response times (left) and response (R) probability (right). X-axis labels indicate the three most recent trials' stimuli, coded as whether they repeated (R) or alternated (A) the preceding trial (e.g., AAA = alternate, alternate, alternate, AAR = alternate, alternate, repeat; the right-most letter in the string indicates the most recent trial). Crosses indicate data. Red circles in the right plot indicate the probability of the stimulus repeating the previous stimulus, which is an important quality check.

more likely to repeat on the current trial. Furthermore, a 'repeat' response would be faster than an 'alternate' response. Now let's fit the model to see the strength of these effects in real data:

```
emc <- make_emc(dat, design=RDM_RM)
emc <- fit(emc)
pp <- predict(emc)
plot_history_effects(dat, pp, degree=2)
```

These results demonstrate that the second-degree recency effects in response probability are fit reasonably well, but some misfit remains in the response times.

Stimulus memory: DDM and drift rate effects

The examples above mimic Miletic et al. (2025) by using an RDM and assuming that stimulus memory causes a threshold (start point) bias. *EMC2* is flexibly designed to implement your own hypotheses. For example, we can propose a WDM instead and hypothesize that drift rates rather than start points are biased. However, as the minimal implementation below demonstrates, this does not appear to fit as well and loses quantitative model comparisons:

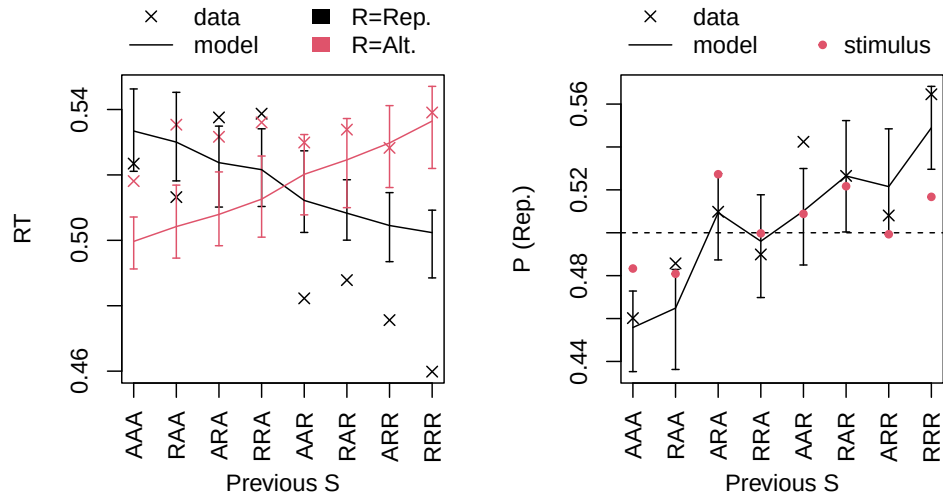


Figure 22: Effects of local stimulus (S) repetition history on response times (left) and response (R) probability (right). X-axis labels indicate the three most recent trials' stimuli, coded as whether they repeated (R) or alternated (A) the preceding trial (e.g., AAA = alternate, alternate, alternate, AAR = alternate, alternate, repeat; the right-most letter in the string indicates the most recent trial). Crosses indicate data, model predictions are indicated by error bars (95% credible intervals), and lines connect medians. Red circles in the right plot indicate the probability of the stimulus repeating the previous stimulus, which is an important quality check.

```
trend_SM_DDM <- make_trend(cov_names='Stim1', kernels='delta',
                           par_names='v', bases='lin',
                           phase="premap", at=NULL)
Smat <- matrix(c(-1,1), nrow=2, dimnames=list(NULL,"dif"))
SM_DDM <- design(model=DDM,
                 data=dat,
                 covariates=c('Stim1'),
                 contrasts=list(S=Smam),
                 formula=list(v ~ S, a ~ 1, t0 ~ 1),
                 constants=c('v.q0'=0, 'v'=0),
                 trend=trend_SM_DDM)

# since v is estimated on the natural scale (in case of the DDM),
# we do not need to change the transformations, and we can use the default priors.
emc_ddm <- make_emc(dat, design=SM_DDM)
emc_ddm <- fit(emc_ddm)
pp_ddm <- predict(emc)

# check quality of fit
plot_history_effects(dat, pp_ddm)

# compare with RDM
compare(sList=list(rdm=emc,
                  ddm=emc_ddm))
```

```
#>      MD wMD   DIC wDIC  BPIC wBPIC EffectiveN meanD Dmean minD
#> rdm -6744  1 -6855   1 -6775    1          79 -6934 -6954 -7014
#> ddm -5274  0 -5353   0 -5277    0          75 -5428 -5447 -5503
```

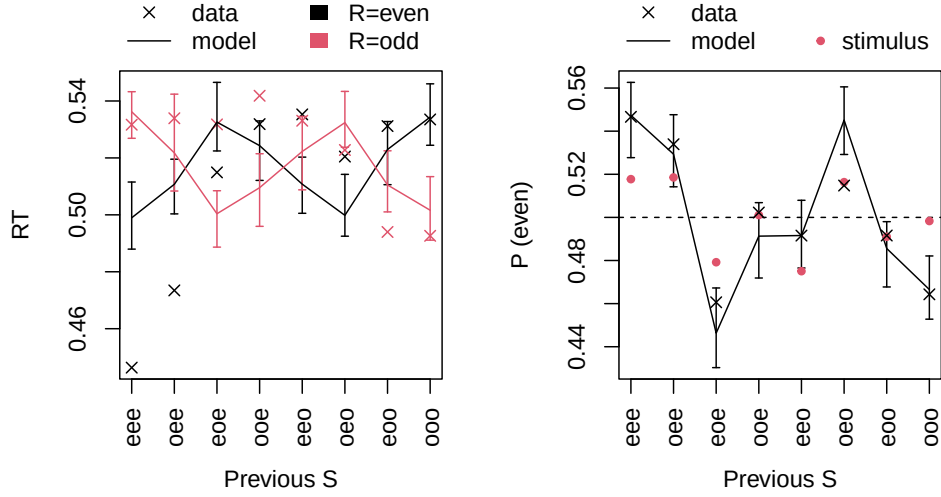


Figure 23: Effects of local stimulus (S) history on response times (left) and response (R) probability (right). X-axis labels indicate the three most recent trials' stimuli (e.g., eee = even, even even, eeo = even, even, odd; the right-most letter in the string indicates the most recent trial). Crosses indicate data, model predictions (in this case, the DDM with a drift rate effect of stimulus memory) are indicated by error bars (95% credible intervals), and lines connect medians. Red circles in the right plot indicate the probability of the stimulus, which is an important quality check.

Mechanisms of dynamics: Accuracy memory

Accuracy memory follows the same logic as above, with a different covariate (errors) and a different parameter (urgency, v). Errors are the inverse of accuracy; they are 0 when the response matches the stimulus, and 1 otherwise. Unlike the presented stimulus, accuracy/error is not a static property of the experimental design but depends on the observed behavior. This is important when simulating new data - the covariate in question is not the *empirically-observed* accuracy on any given trial, but the *newly-simulated* accuracy.

For this reason, it is useful to have *EMC2* calculate accuracy with a function rather than specifying it in the dataframe (in which case it would be treated as a static experimental factor). The function is applied when generating posterior predictives, ensuring that the accuracy of the *simulated* data is used as a covariate.

```

AM_trend <- make_trend(cov_names='error',
                      kernels='delta',
                      par_names='v',
                      bases='lin',
                      phase="premap")
RDM_AM <- design(model=RDM,
                data=dat,
                contrast=list(lM=ADmat, lR=ADmat),
                functions=list(error=function(x) x$S!=x$R),
                matchfun=function(d) d$S==d$lR,
                transform=list(lower=c(v.alpha=0.01)),
                formula=list(B ~ 1, v ~ lM, t0 ~ 1, s~lM),
                trend=AM_trend,
                constants=c('v.q0'=0, 's'=1),
                report_p_vector=FALSE)

# NB1: we allow within-trial noise s to vary with accumulator
# match (correct/incorrect), which will allow us to capture the
# relative speed of errors.

# NB2: in this case, we are not estimating v on the natural scale:
mapped_pars(RDM_AM)
# This is because the AM mechanism can push drift rates on individual trials
# to negative values when applied on the natural scale.
# Since the RDM has no known analytic likelihood for negative drift rates these
# are excluded by the RDM model using its bound attribute (see RDM()$bound), but
# this can lead to difficulties sampling. Hence, the effect of errors on
# urgency as we implement it here, is not strictly linear -- it is linear on
# the log-scale.
# The cognitive interpretation remains similar.

emc <- make_emc(dat, design=RDM_AM)
emc <- fit(emc)
credint(emc)

```

```

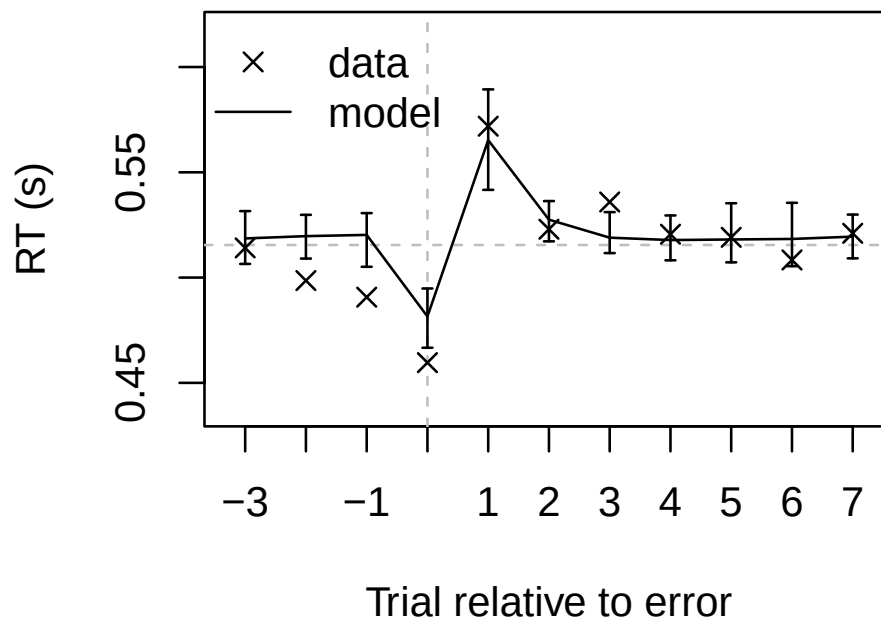
#> $v
#> lM
#> TRUE : exp(v_t + 0.5 * v_lMd)
#> FALSE : exp(v_t - 0.5 * v_lMd)
#> Trends:
#> v_t = v + v.w * q[i]
#>
#> $s
#> lM
#> TRUE : exp(s + 0.5 * s_lMd)
#> FALSE : exp(s - 0.5 * s_lMd)
#> Time difference of 13.97568 mins
#> $mu
#>
#> 2.5% 50% 97.5%
#> B 0.944 1.262 1.491
#> v 0.583 1.119 1.763
#> v_lMd 1.102 2.506 3.547
#> t0 -2.013 -1.717 -1.347
#> s_lMd -0.497 -0.351 -0.137
#> v.w -0.244 -0.152 -0.053
#> v.alpha 0.350 0.907 1.561

```

Accuracy memory can explain error-related effects such as post-error slowing. To visualize, we can generate posterior predictives and then use a convenience function to plot mean RT as a function of trial number

relative to an error. As mentioned above, since the covariate in this case depends on observed behavior, we need to simulate data and determine the covariate one trial at a time. *EMC2* detects whether the covariate is `rt`, `R`, or the output of one of the functions provided to design. If it is, it automatically simulates data trial-by-trial. Because vectorisation is not possible in this case, this way of simulating data is much slower.

```
pp <- predict(emc)
# this is a convenience function that is not part of EMC2. For help, inspect the
# first lines after calling plot_error_effects (omitting the parentheses),
# which document its use
plot_error_effects(dat, pp)
```



Like in the case of stimulus memory, the accuracy memory mechanism can be adjusted to other EAMs like the DDM, and it can affect other parameters. The DDM does not have an urgency mechanism, but we could, for example, allow AM to affect thresholds. This trend can be specified as post-transform since the thresholds are the same across all design cells. So in this case, we can sample thresholds on the log scale, then transform to the natural scale, and then apply the trend. The example below demonstrates its implementation, illustrates the poor fit of the error-related effects, and shows how it loses model comparisons against the previous model.

```

trend_AM_DDM <- make_trend(cov_names='error',
                           kernels='delta',
                           par_names='a',
                           bases='lin',
                           phase='posttransform',
                           at=NULL)
Smat <- matrix(c(-1,1), nrow=2,dimnames=list(NULL,'dif'))
DDM_AM <- design(model=DDM,
                 data=dat,
                 functions=list(error=function(x) x$S!=x$R),
                 contrasts=list(S=Smat),
                 formula=list(v ~ S, a ~ 1, t0 ~ 1),
                 constants=c('a.q0'=0),
                 trend=trend_AM_DDM)

emc_ddm_am <- make_emc(dat, design=DDM_AM, compress=FALSE)
emc_ddm_am <- fit(emc_ddm_am)
pp_ddm_am <- predict(emc_ddm_am)
plot_history_effects(dat, pp_ddm_am)
compare(list(rdm=emc, ddm=emc_ddm_am))

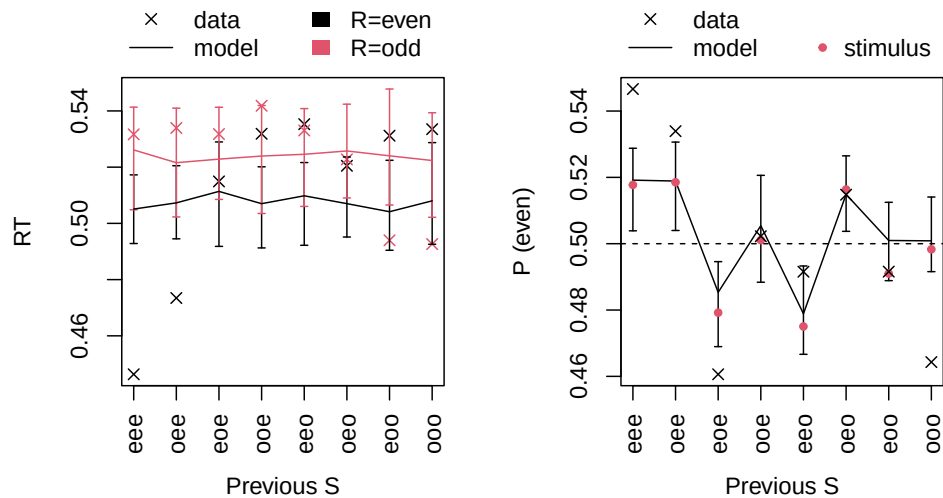
```

```

#>
#> Sampled Parameters:
#> [1] "v"      "v_Sdif" "a"      "t0"     "a.w"    "a.alpha"
#>
#> Design Matrices:
#> $v
#>   S v v_Sdif
#> even 1    -1
#> odd 1     1
#>
#> $a
#> a
#> 1
#>
#> $t0
#> t0
#> 1
#>
#> $a.w
#> a.w
#> 1
#>
#> $a.q0
#> a.q0
#> 1
#>
#> $a.alpha
#> a.alpha
#> 1
#>
#> $s
#> s
#> 1
#>
#> $st0
#> st0
#> 1

```

```
#>
#> $sv
#> sv
#> 1
#>
#> $SZ
#> SZ
#> 1
#>
#> $Z
#> Z
#> 1
```

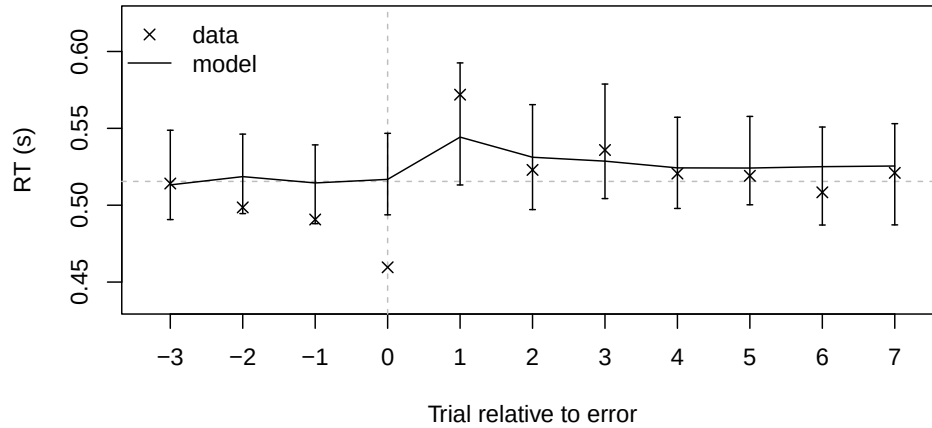


```
#>      MD wMD   DIC wDIC  BPIC wBPIC EffectiveN meanD Dmean minD
#> rdm -6905  1 -7041   1 -6962    1         79 -7120 -7133 -7200
#> ddm -5297  0 -5379   0 -5290    0         89 -5468 -5498 -5557
```

However, this again does not appear to fit as well qualitatively, and loses model comparisons:

```
emc <- get(load('./samples/ds1_AMa_DDM.RData'))
pp_ddm <- predict(emc, n_cores=40, conditional_on_data=FALSE)
save(pp_ddm, file='./samples/ds1_AMa_DDM_pps.RData')
```

```
plot_error_effects(dat, pp_ddm)
```



```
# and loses model comparisons again
compare(sList=list(rdm=get(load('./samples/ds1_AM.RData')),
                  ddm=get(load('./samples/ds1_AMa_DDM.RData'))))
#>      MD wMD  DIC wDIC BPIC wBPIC EffectiveN meanD Dmean minD
#> rdm -6891  1 -7016  1 -6921  1      95 -7111 -7122 -7207
#> ddm -5297  0 -5392  0 -5317  0      75 -5467 -5499 -5543
```

Special case: Fluency memory

Fluency memory is a formalization of the idea that participants adjust their behavior according to perceived difficulty. To estimate perceived difficulty, we rely on a few simplifying assumptions: participants can quantify (1) their thresholds, and (2) their response time. In race models, the ratio of threshold over response time approximates mean speed, which can be used as a proxy for difficulty, as processing efficiency will be lower for difficult trials relative to easy trials. Formally, $Q_{FM,t+1} = Q_{FM,t} + \alpha_{FM}(b_t/rt_t - Q_{FM,t})$.

Here, we implement this using a custom kernel. Note that when we load a custom-kernel emc object from disk we need to reinitialize C pointers (see `?fix_custom_kernel_pointers`).

```

tf <- tempfile(fileext='.cpp')
writeLines(c(
  "// [[Rcpp::depends(EMC2)]]",
  "#include <Rcpp.h>",
  "#include \"EMC2/userfun.hpp\"",
  "",
  "// Example: two params (q0, alpha) and three inputs (rt, B, weight)",
  "Rcpp::NumericVector custom_kernel(Rcpp::NumericMatrix kernel_pars, ",
  "                                Rcpp::NumericMatrix input) {",
  "  // assume kernels_pars(q0, alpha) and inputs(rt, B, weight)",
  "  int n = input.nrow();",
  "  Rcpp::NumericVector q(n);",
  "  Rcpp::NumericVector B_t(n);",
  "  Rcpp::NumericVector pe(n);",
  "  // B_t = B + w*Q_t for trial 1",
  "  q[0] = kernel_pars(0,0);",
  "  B_t[0] = input(0, 1) + input(0, 2)*q[0];",
  "  for (int i = 1; i < n; ++i) {",
  "    // 'prediction error' = B_t/rt_t - Q_t",
  "    pe[i-1] = (B_t[i-1] / input(i-1,0)) - q[i-1];",
  "    // update: Q_t = Q_t-1 + alpha_t-1 * pe_t-1",
  "    q[i] = q[i-1] + kernel_pars(i-1,1) * pe[i-1];",
  "    // Apply base to B to get the correct threshold for ",
  "    // next trial (needed for PE calculation)",
  "    B_t[i] = input(i,1) + input(i,2) * q[i];",
  "  }",
  "  return q;",
  "}",
  "",
  "// Export pointer maker for registration",
  "// [[Rcpp::export]]",
  "SEXP EMC2_make_custom_kernel_ptr();",
  "EMC2_MAKE_PTR(custom_kernel)"
), tf)

```

```

ct <- register_trend(
  trend_parameters=c("q0", "alpha"),
  file=tf,
  transforms=c(q0="identity", alpha="pnorm"),
  base="lin"
)

```

```

trend_FM=make_trend(par_names='B',
  cov_names=c('rt'),
  kernels='custom',
  par_input=list(c('B', 'B.w')),
  bases=NULL, # uses ct$base (here: lin)
  custom_trend=ct)

```

```

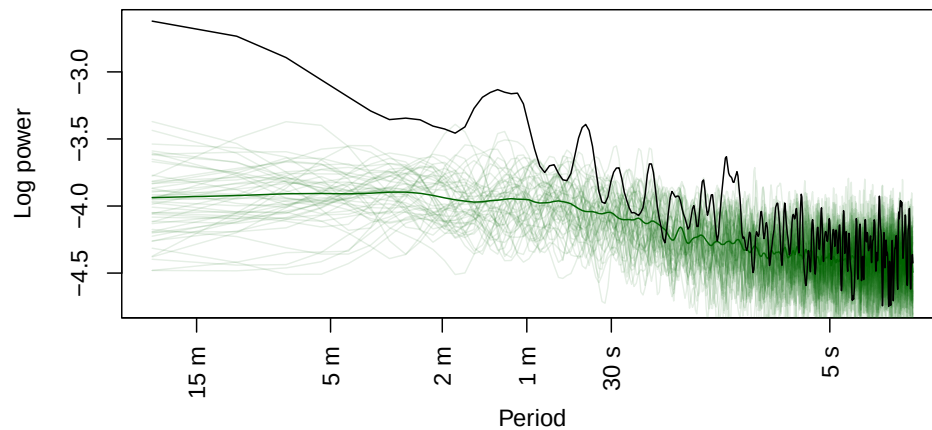
ADmat <- matrix(c(-.5,.5), ncol=1, dimnames=list(NULL,'d'))
dat <- EMC2::add_trials(dat)
design_FM <- design(model=RDM,
  data=dat,
  covariates=c('rt'),
  contrast=list(LM=ADmat),
  matchfun=function(d) d$S==d$1R,
  formula=list(B ~ 1, v ~ LM, t0 ~ 1, s~LM),
  transform=list(func=c(B='identity'),
    lower=c(B.alpha=0.05)),
  constants=c('B.q0'=3, # NB1
    's'=log(1)),
  trend=trend_FM)

```

*# NB1: Why q0=3? When fitting a static RDM,
 # the mean fit threshold over mean RT ratio is very roughly 3
 # in this dataset. It serves as a reasonable place to start*

Fluency memory provides an explanation for the slower fluctuations in the observed response times. One useful way to visualize these is by creating a power spectrum of the RT data. We provide a convenience function that does this for the data and the posterior predictives of the estimated model:

```
# convenience function to plot the spectrum
plot_spectrum(dat=dat, pp=pp, trial_duration=1.265)
```



```
# mean trial duration of this task was 1.265 s,  
# which we use to determine the periods
```

The spectrum shows that fluency can account for the much of the power in fluctuations that are moderate to fast, but not the slowest fluctuations. Note that we supplied the mean trial duration to determine the periods, which are then plotted on the x axis. If this is not supplied the x axis is instead $\log(\text{frequency})$.

- Jones, Matt, Tim Curran, Michael C. Mozer, and Matthew H. Wilder. 2013. “Sequential Effects in Response Time Reveal Learning Mechanisms and Event Representations.” *Psychological Review* 120 (3): 628–66. <https://doi.org/10.1037/a0033180>.
- Miletić, Steven, Niek Stevenson, Ami Eidels, Dora Matzke, Birte Forstmann, and Andrew Heathcote. 2025. “Explaining Multi-Scale Choice Dynamics.” *Psychological Review*. <https://doi.org/10.1037/rev0000581>.
- Stevenson, Niek, Michelle Donzallaz, Reilly James Innes, Birte Forstmann, Dora Matzke, and Andrew Heathcote. 2024. “EMC2: An R Package for Cognitive Models of Choice.” <https://doi.org/10.31234/osf.io/2e4dq>.
- Wagenmakers, Eric-Jan, Simon Farrell, and Roger Ratcliff. 2004. “Estimation and Interpretation of $1/f\alpha$ Noise in Human Cognition.” *Psychonomic Bulletin & Review* 11 (4): 579–615. <https://doi.org/10.3758/BF03196615>.